Cloud Databases: A Systematic Meta-Survey of Current Challenges, Emerging Paradigms, and Future Research Directions

Abdulhakeem Ibrahim[a,*]

[a]School of Computer Science and Informatics, De Montfort University, Leicester, United Kingdom

ARTICLE INFO

Keywords:
cloud databases
systematic meta-survey
cloud-native databases
database-as-a-service
data lakehouse
AI4DB
survey of surveys

ABSTRACT

Cloud databases have become the backbone of modern data-driven applications, enabling organizations to store, process, and analyze data at unprecedented scale and elasticity. A search window spanning January 1999 to July 2026 surfaces a body of cloud-database survey literature that, in practice, begins around 2009; from it, 84 survey papers satisfying systematic inclusion and quality criteria examine various facets of cloud database technology—spanning cloud-native architectures, hybrid transactional and/or analytical processing (HTAP), NoSQL/NewSQL systems, vector and graph databases, AI-for-database optimization (AI4DB), data lakehouses, time-series databases, cloud database migration, and edge–cloud data management. However, the challenges, open problems, and research directions identified in these surveys remain fragmented across disparate communities and publication venues. This paper presents, to the best of our knowledge, the first *systematic tertiary study* of cloud database research—a survey of surveys that synthesises findings from systematically selected survey papers using a protocol grounded in the Kitchenham–Charters guidelines and reported in line with PRISMA 2020. Because inclusion is restricted to full texts we could retrieve and quality-assess, and roughly two-thirds of otherwise-eligible reports were not retrievable through open-access channels, the synthesis is best read as a broad, transparent map rather than an exhaustive census. The scattered landscape of cloud database challenges is organized into a structured, dual-theme taxonomy: (1) *eighteen cross-cutting challenges* that transcend individual system categories, including scalability, consistency trade-offs, security, cost optimization, AI–database convergence, and sustainability; and (2) *five lifecycle-phase challenges* spanning design, deployment, operation, optimization, and evolution of cloud database systems. The convergence of cloud databases with large language models (LLMs) is further examined as an illustrative case study, and a consolidated set of open research directions is distilled from the cross-survey synthesis. The result is a single, thematically organised map of roughly fifteen years of cloud database survey literature—one that newcomers can use to orient themselves and that system builders and researchers can use to locate the open problems where new work is most needed.

1. Introduction

The cloud database market has experienced explosive growth over the past decade, and cloud offerings now dominate the most widely tracked DBMS popularity rankings [1]. This growth reflects a fundamental shift in how organizations manage data: from on-premises, monolithic database servers to elastic, globally distributed, cloud-native data platforms that can scale storage and compute independently on demand [2, 3]. Major cloud providers—Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and Alibaba Cloud—now offer dozens of specialized database services, from relational engines like Amazon Aurora [2] and Azure SQL [4] to purpose-built systems for documents, graphs, time series, key-value pairs, and, most recently, vector embeddings for AI workloads [5].

This rapid evolution has attracted significant research attention. A scripted search across four scholarly indexing services (OpenAlex, arXiv, Crossref, and Semantic Scholar) covering January 1999 to July 2026 identifies **84 survey papers** meeting the inclusion and quality criteria, each examining a particular facet of cloud database technology. These surveys span a wide spectrum of topics: cloud-native database architectures [6], hybrid transactional/analytical processing (HTAP) [7], graph databases [8], vector database management systems [5], AI-driven database optimization (AI4DB) [9], data lakehouses [10], NoSQL and NewSQL systems [11, 12], edge and fog database systems [13], and many more.

*Corresponding author.
hakeem.ibrahim@dmu.ac.uk (A. Ibrahim)
ORCID(s): 0000-0002-1104-5314 (A. Ibrahim)

Despite this wealth of individual surveys, the field lacks a *meta-level synthesis*—a comprehensive, systematic effort to consolidate the challenges, open problems, and research directions scattered across these surveys into a unified framework. Each survey naturally focuses on its own domain, leading to a fragmented understanding of the cloud database landscape as a whole. Cross-cutting challenges such as cost optimization, security, consistency trade-offs, and AI–database convergence are discussed in many surveys but from different angles, without a common organizing framework.

1.1. Contributions and novelty

This paper addresses the gap by presenting what is, to the best of our knowledge, the **first systematic tertiary study of cloud database research**—a survey of surveys. (We adopt Kitchenham's term *tertiary study* for a systematic review whose primary studies are themselves systematic reviews or surveys; we searched for prior tertiary studies of cloud databases as part of the selection process and found none.) A rigorous systematic literature review (SLR) protocol based on the Kitchenham–Charters guidelines [14] is applied to identify, evaluate, and synthesize cloud database surveys. The contributions of this work are as follows:

C1 Systematic identification and classification. Four scholarly APIs are searched with a fully scripted, reproducible protocol over the window January 1999 to July 2026, yielding 85 quality-assessed survey papers on cloud databases, classified by topic, scope, methodology, and venue.

C2 Dual-theme challenge taxonomy. The scattered challenges are organized into a structured taxonomy with two complementary themes:

- *Theme 1: Eighteen cross-cutting challenges* that recur across multiple survey domains, including scalability, consistency, security, cost modeling, AI–database convergence, and sustainability.
- *Theme 2: Five lifecycle-phase challenges* that map to the stages of designing, deploying, operating, optimizing, and evolving cloud database systems.

C3 LLM–database convergence analysis. The rapidly emerging intersection of large language models (LLMs) and cloud databases is examined as an illustrative case study, covering text-to-SQL, vector databases for retrieval-augmented generation (RAG), and LLM-based database administration.

C4 Research agenda. A consolidated, cross-cutting research agenda is distilled from the synthesis, grouping the open problems that recur most widely across the surveyed literature.

C5 Knowledge engineering perspective. Cloud database challenges are explicitly connected to data and knowledge engineering concerns—including knowledge graphs, ontologies, metadata management, and schema evolution—making this survey particularly relevant to the *Data & Knowledge Engineering* community.

1.2. Positioning relative to prior surveys

Several of the individual surveys in our corpus are themselves broad, but each remains bounded to a single domain and reviews *primary* studies; none takes surveys as its unit of analysis or spans the full cloud-database landscape. Table 1 contrasts this tertiary study with three of the most comprehensive domain surveys it includes. The distinction is one of altitude: where a domain survey answers "what is known about cloud-native engines / NoSQL stores / DBMS performance," this study answers "what do the cloud-database surveys, collectively, agree and disagree on, and where are the gaps."

1.3. Research questions

This meta-survey is guided by five research questions, summarized in Table 2 and elaborated in Section 4.

1.4. Paper organization

The remainder of this paper is organized as shown in Figure 1. Section 2 motivates the importance of cloud databases from five complementary perspectives. Section 3 establishes key definitions and distinctions—from cloud-hosted to cloud-native databases, architectural paradigms, data models, and the data platform continuum. Section 4 details the systematic review methodology, including search strategy, selection criteria, quality assessment, and results. Section 5 presents the core contribution: a comprehensive discussion of challenges and research directions organized by

Table 1
Positioning of this tertiary study against representative broad surveys within its own corpus.

Study	Scope	Method	# studies	Organizing structure
Li et al. (cloud-native) [6]	Cloud-native engines	Narrative survey	primary systems	Architectural components
Davoudian et al. (NoSQL) [11]	NoSQL stores	Narrative survey	primary systems	Data models & features
Taipalus (DBMS perf.) [15]	DBMS benchmarking	SLR	primary studies	Measurement practice
This study	**All cloud DB domains**	**Systematic tertiary**	**84 surveys**	**18 cross-cutting + 5 lifecycle**

Table 2
Research questions guiding the meta-survey.

ID	Research Question
RQ1	What is the landscape of cloud database surveys published between 1999 and 2026, in terms of topic coverage, methodology, and publication venue?
RQ2	What are the major cross-cutting challenges identified across cloud database surveys, and how do they relate to one another?
RQ3	How do cloud database challenges map to the lifecycle phases of designing, deploying, operating, optimizing, and evolving database systems?
RQ4	What is the current state and future trajectory of the convergence between cloud databases and AI/ML technologies, particularly large language models?
RQ5	What are the most pressing open research directions, and which of them recur most widely across the surveyed literature?

cross-cutting themes and lifecycle phases. Section 6 examines the convergence of cloud databases and LLMs. Section 7 concludes. Appendix A provides the complete catalog of surveyed papers.

Figure 1: The organization of this meta-survey paper.

2. Why Cloud Databases Matter—A Multi-Perspective Motivation

Understanding the significance of cloud databases requires examining the topic from multiple viewpoints. In this section, the need for a meta-survey is motivated by discussing five complementary perspectives (Figure 2), each revealing why the scattered state of survey knowledge is problematic and why a unified synthesis is timely.

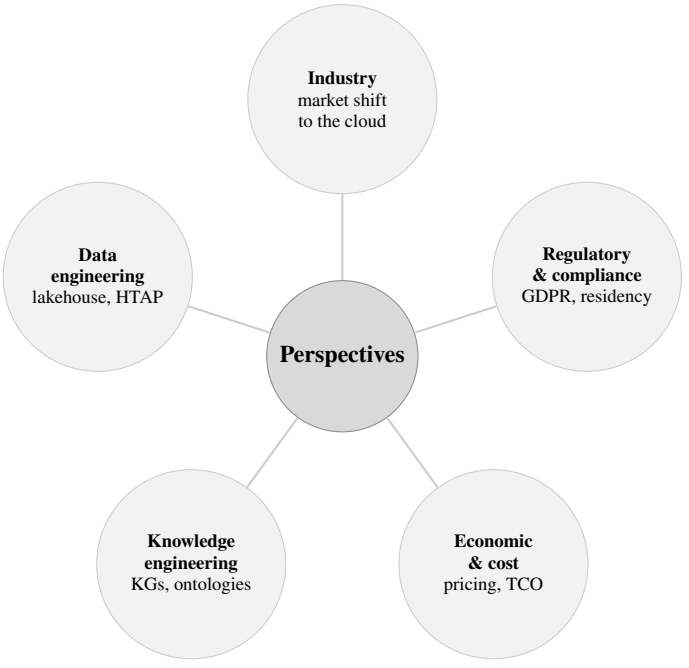

Figure 2: The five complementary perspectives motivating a systematic meta-survey of cloud databases.

2.1. Industry perspective

Two structural trends are widely reported. First, cloud-based systems now occupy the top of the most-tracked DBMS popularity rankings, and their share continues to grow at the expense of on-premises engines [1]. Second, Gartner's widely cited 2019 projection that three quarters of all databases would be deployed on or migrated to a cloud platform by 2022 anticipated this shift [16]. This shift is driven by digital transformation initiatives across industries, from financial services adopting cloud databases for real-time fraud detection to healthcare organizations leveraging cloud data platforms for genomics and precision medicine.

The vendor landscape has also diversified significantly. Beyond the hyperscale cloud providers (AWS, Azure, GCP), specialized cloud database vendors such as Snowflake [3], Databricks [17], CockroachDB [18], PlanetScale, and Neon have gained substantial market share. China's cloud database ecosystem—including Alibaba PolarDB [19], TiDB [20], and OceanBase [21]—has matured rapidly. This fragmentation makes it difficult for practitioners to navigate the landscape without a systematic overview.

2.2. Data engineering perspective

Modern data architectures have evolved from simple transactional databases to complex, multi-layered data ecosystems. The emergence of the *data lakehouse* paradigm [17] exemplifies this evolution, combining the flexibility of data lakes with the governance and performance of data warehouses. Systems like Databricks Delta Lake, Apache Iceberg, and Apache Hudi have created a new "open table format" layer that bridges batch and streaming analytics [10].

Simultaneously, the rise of *HTAP* (Hybrid Transactional/Analytical Processing) systems [7] challenges the traditional separation between OLTP and OLAP workloads. Systems such as TiDB [20], SAP HANA [22], and Google AlloyDB promise unified engines that handle both transaction processing and analytical queries. Understanding the trade-offs, maturity, and open challenges of these paradigms requires synthesizing insights across multiple survey papers.

2.3. Knowledge engineering perspective

Cloud databases play a crucial role in knowledge engineering, making this topic directly relevant to the *Data & Knowledge Engineering* (DKE) community. Several connections are worth highlighting:

- **Knowledge graphs in the cloud.** Large-scale knowledge graphs (KGs) such as Google Knowledge Graph, Wikidata, and enterprise KGs are increasingly stored and queried using cloud-hosted graph databases (e.g., Amazon Neptune, Azure Cosmos DB with Gremlin API, Neo4j Aura) [8, 23, 24]. The scalability, consistency, and query optimization challenges of cloud graph databases directly impact KG applications.
- **Ontology-mediated data access.** Cloud environments with heterogeneous data sources benefit from ontology-based data integration (OBDI) techniques [25], where ontologies provide a unified conceptual layer over distributed cloud databases.
- **Metadata and schema management.** Cloud data catalogs and metadata management systems (e.g., AWS Glue Data Catalog, Apache Atlas) implement knowledge engineering principles for organizing, discovering, and governing data assets [26].
- **AI–knowledge engineering convergence.** The integration of machine learning with database systems (AI4DB) [9] draws on knowledge engineering concepts such as learned models for query optimization, workload characterization, and automated tuning.

2.4. Economic and cost perspective

Cloud databases introduce novel economic dynamics compared to on-premises deployments. The shift from capital expenditure (CapEx) to operational expenditure (OpEx) models, combined with pay-per-use pricing and complex multi-dimensional billing (compute, storage, I/O, network egress, backups), creates significant challenges in cost prediction and optimization [27, 28].

Research has shown that cloud database costs can vary by orders of magnitude depending on configuration choices, query patterns, and pricing model selection [27]. The total cost of ownership (TCO) analysis becomes particularly complex for multi-cloud or hybrid cloud deployments. Several surveys touch on cost-related challenges [6, 29], but a consolidated analysis is lacking.

Table 3
Comparison of cloud database deployment models.

Characteristic	Cloud-Hosted	Cloud-Native	Cloud-Born
Definition	Traditional DBMS deployed on cloud VMs	DBMS redesigned to exploit cloud infrastructure	DBMS designed from scratch for the cloud
Architecture	Monolithic, tightly coupled	Disaggregated compute/storage	Fully disaggregated, microservice-based
Elasticity	Manual vertical scaling; horizontal read replicas	Automatic scaling (horizontal)	Instant, fine-grained scaling
Examples	MySQL on EC2, Oracle on Azure VM	Amazon Aurora [2], Azure SQL [4]	Snowflake [3], Neon, CockroachDB [18]
State sharing	Shared-nothing (typically)	Shared-storage	Disaggregated, shared cloud services
Multi-tenancy	VM-level isolation	Engine-level isolation	Native multi-tenant design

2.5. Regulatory and compliance perspective

Data sovereignty regulations such as the European Union's General Data Protection Regulation (GDPR) [30], China's Data Security Law, and sector-specific requirements (e.g., HIPAA for healthcare, PCI-DSS for payment data) impose strict constraints on how and where cloud databases store and process data [31].

Multi-region deployment, data residency requirements, cross-border data transfer restrictions, and audit trail obligations all impact cloud database architecture and operations. The compliance landscape interacts with technical challenges such as geo-distributed consistency, encryption, access control, and data lineage—topics discussed in various surveys but rarely addressed in an integrated manner.

3. From Cloud Databases to Cloud-Native Data Platforms

Before analyzing the surveyed literature, key concepts, definitions, and distinctions are established that form the vocabulary of cloud database research.

3.1. Cloud-hosted vs. cloud-native vs. cloud-born databases

The term "cloud database" encompasses a spectrum of architectures with fundamentally different design philosophies. Three categories are distinguished, compared in Table 3:

Cloud-hosted databases are traditional database engines (e.g., PostgreSQL, MySQL, Oracle) deployed on cloud virtual machines. They benefit from cloud infrastructure (networking, storage volumes, automated backups) but do not exploit cloud-native primitives such as elastic compute pools or disaggregated storage [6].

Cloud-native databases are existing or new engines that have been substantially re-architected to leverage cloud services. Amazon Aurora [2] exemplifies this approach: it re-implements the storage layer of MySQL/PostgreSQL to use a distributed, log-structured storage service while keeping the SQL engine largely intact. Microsoft's Socrates [4] takes SQL Server and disaggregates its components across cloud microservices.

Cloud-born databases—a term we use here for systems designed entirely from the ground up for cloud deployment, with no legacy on-premises heritage—were pioneered for data warehousing by Snowflake [3], while this model for data warehousing, while CockroachDB [18] and Neon represent cloud-born approaches for transactional and PostgreSQL-compatible workloads, respectively.

3.2. A brief history: from early outsourced databases to cloud-native platforms (1999–2015)

The systematic search window of this meta-survey opens in January 1999, yet the earliest survey satisfying all inclusion and quality criteria dates from 2009 (Table 12)—a finding that itself traces the field's history. The idea of consuming database functionality as an outsourced service predates the cloud era: the "database-as-a-service" model was articulated in the early 2000s around outsourced, encrypted query processing, and the security and confidentiality questions it raised dominated the first wave of survey activity [32, 33]. Commercial cloud databases arrived with Amazon SimpleDB and Google Bigtable [34], whose early comparative assessments [35] and elasticity

measurements [36] mark the transition from outsourced databases to genuinely cloud-resident data stores. The NoSQL wave of 2009–2013 [37, 38] multiplied data models and triggered the first generation of comparative NoSQL surveys, while practitioner-oriented overviews began mapping the emerging cloud database service landscape [39, 40, 41]. By the mid-2010s, quality-attribute-driven selection studies [42] and consolidated DBaaS security taxonomies [33] signal a maturing field—setting the stage for the cloud-native re-architecting era that the majority of the catalog documents from 2016 onward.

3.3. Database-as-a-Service (DBaaS) spectrum

Database-as-a-Service (DBaaS) represents the fully managed end of the cloud database spectrum, where the cloud provider handles provisioning, patching, scaling, backup, and high availability [43, 44, 45, 46, 39]. The DBaaS model has evolved along several dimensions:

- **Managed databases**: Provider-managed instances of open-source or commercial engines (e.g., Amazon RDS, Azure Database for PostgreSQL).
- **Serverless databases**: Fully elastic databases that scale to zero when idle and automatically provision resources based on demand (e.g., Amazon Aurora Serverless, Azure Cosmos DB serverless, Neon) [29].
- **Autonomous databases**: Self-driving databases that use ML to automate tuning, indexing, partitioning, and anomaly detection (e.g., Oracle Autonomous Database) [47].

3.4. Architectural paradigms

Cloud database architectures have evolved through three major paradigms, illustrated in Figure 3:

Shared-nothing architectures partition data across independent nodes, each with its own CPU, memory, and disk. Traditional distributed databases such as Cassandra follow this model; early Spanner [48] is often cited here for its shared-nothing compute partitioning, though it already layered this over the shared Colossus file system and so sits closer to the shared-disk boundary than a pure shared-nothing store. While horizontally scalable, resharding and rebalancing are expensive.

Shared-disk (cloud storage) architectures decouple compute from storage, allowing compute nodes to share a common cloud storage layer. Amazon Aurora [2] and Snowflake [3] exemplify this paradigm, enabling independent scaling of compute and storage.

Fully disaggregated architectures further decompose the database engine into independently scalable microservices—query processing, transaction management, logging, metadata management, and storage [4]. Microsoft Socrates and emerging systems like PolarDB [19] pursue this vision.

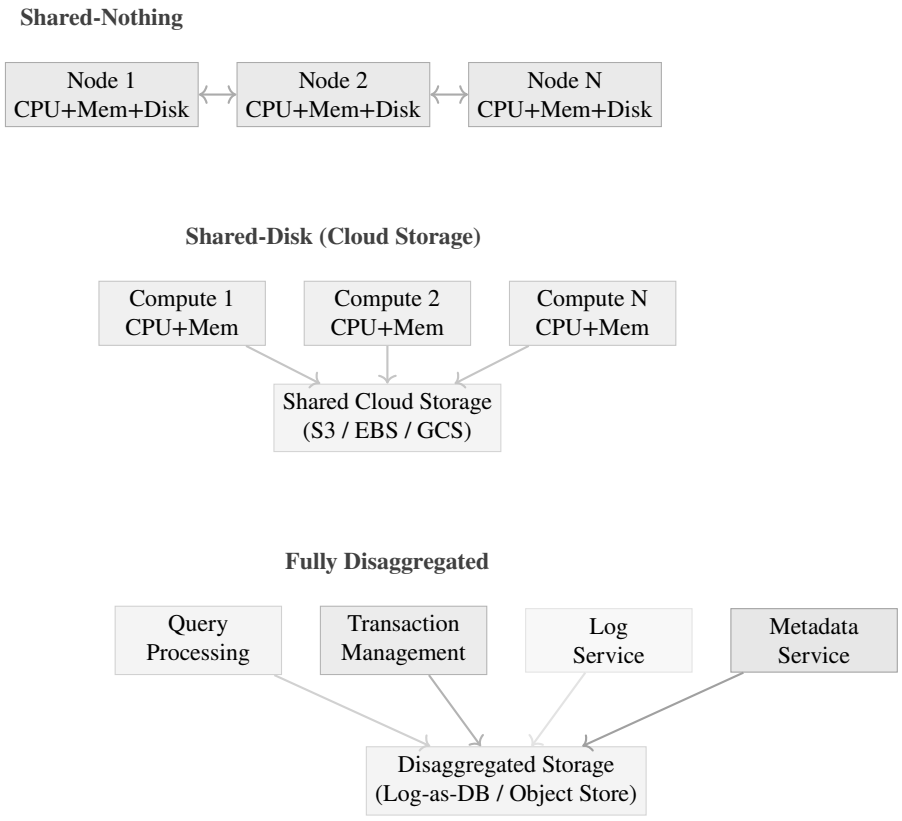

Figure 3: Evolution of cloud database architectures: from shared-nothing clusters to shared-disk (cloud storage) to fully disaggregated microservice architectures.

3.5. Data models in the cloud

Cloud platforms support a diverse ecosystem of data models, each optimized for different workload characteristics [49, 50, 38, 51, 52]; early comparisons of the first-generation cloud stores such as SimpleDB and Bigtable already highlighted this model diversity [35]. Table 4 compares the major data models available in cloud database services.

The trend toward *multi-model databases* [53] reflects the desire to avoid managing separate database systems for different data models. Azure Cosmos DB, for instance, supports document, graph, key-value, column-family, and table APIs through a single backend. However, multi-model integration introduces challenges in query optimization, consistency semantics, and performance trade-offs [53].

3.6. The data platform continuum: warehouses, lakes, and lakehouses

The cloud data management landscape has converged around three complementary paradigms, illustrated in Figure 4:

Table 4
Cloud database data model landscape.

Data Model	Strengths	Cloud Examples	Typical Use Cases
Relational	ACID, SQL, mature ecosystem	Aurora, Cloud SQL, Azure SQL	OLTP, ERP, financial systems
Document	Schema flexibility, nested data	MongoDB Atlas, Cosmos DB, Firestore	Content management, catalogs, user profiles
Key-Value	Ultra-low latency, high throughput	DynamoDB, Redis Cloud, Memorystore	Caching, session management, real-time
Wide-Column	High write throughput, time-series	Bigtable, Cassandra (Astra), ScyllaDB	IoT, telemetry, event logging
Graph	Relationship traversal, pattern matching	Neptune, Neo4j Aura, Cosmos DB (Gremlin)	Social networks, fraud detection, KGs
Vector	Similarity search, ANN indexing	Pinecone, Weaviate, Milvus, pgvector	RAG, recommendation, image search
Time-Series	Temporal queries, downsampling	TimescaleDB, InfluxDB Cloud, Timestream	Monitoring, financial ticks, sensor data
Multi-Model	Multiple models in one engine	Cosmos DB, ArangoDB, SurrealDB	Polyglot persistence, unified access

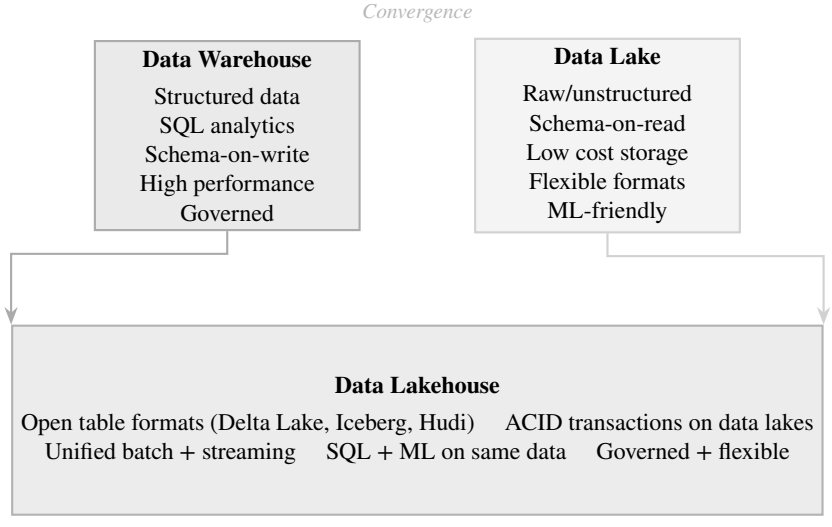

Figure 4: The data platform continuum: data warehouses and data lakes converge into the data lakehouse paradigm.

Data warehouses (e.g., Snowflake [3], BigQuery [54], Redshift) provide high-performance SQL analytics over structured, curated data with strong governance and schema enforcement.

Data lakes (e.g., Amazon S3 + Athena, Azure Data Lake Storage) offer cost-effective storage for raw, semi-structured, and unstructured data, enabling schema-on-read flexibility for data science and ML workloads.

Data lakehouses [17] combine the best of both paradigms by adding ACID transactions, schema enforcement, and data governance to open data lake storage through open table formats such as Delta Lake, Apache Iceberg, and Apache Hudi [10, 55].

Table 5
Scholarly APIs searched (5 July 2026) and raw record yields. The OpenAlex sweep issued one query per cloud-database term (18 queries), each conjoined with the survey-indicator block of the canonical search string below; the arXiv sweep combined the `cs.DB` category with title-level survey indicators.

API	Query scope	Records
OpenAlex	Title + abstract; 18 term-queries × survey-indicator block; 1999–2026; article/review/preprint; English	3,684
arXiv API	`cat:cs.DB` × title survey-indicators	356
Crossref (suppl.)	Bibliographic relevance queries; journal articles from 1999	600
Semantic Scholar (suppl.)	Bulk keyword search; 1999–2026	3,035
Total raw records		7,675
After deduplication (DOI, arXiv id, normalized-title matching)		6,613

4. Systematic Review Planning and Execution

The systematic literature review (SLR) guidelines established by Kitchenham and Charters [14], which have been widely adopted in software engineering and computer science [56, 57, 58, 59], are followed. This protocol prescribes a three-phase process: (1) *planning*, in which research questions, search strategy, and inclusion/exclusion criteria are defined; (2) *conducting*, in which studies are systematically identified, selected, and their data extracted; and (3) *reporting*, in which the synthesized findings are documented and validated through quality assessment. The selection process is additionally reported using the PRISMA 2020 statement [60], the de facto standard for transparent reporting of systematic reviews.

4.1. Search strategy

The search was executed as a fully scripted, reproducible sweep over four scholarly indexing services via their public APIs on 5 July 2026, as detailed in Table 5. OpenAlex and the arXiv API served as the primary sources; Crossref and Semantic Scholar were queried as supplementary sources to catch records missed by the primary indexes. The complete search scripts, the per-query result log, and every screening decision are archived alongside the manuscript (see the Data availability statement), so the identification stage can be re-executed verbatim.

The canonical search string combines cloud database terminology with survey-indicating terms; it was applied natively by the OpenAlex queries and re-applied uniformly to all records (including the relevance-ranked supplementary sources) as an automated eligibility marking step:

```
("cloud database" OR "cloud-native database" OR "DBaaS" OR "database-as-a-service" OR
"serverless database" OR "HTAP" OR "NewSQL" OR "data lakehouse" OR "vector database" OR
"AI4DB" OR "learned index" OR "NoSQL" OR "distributed database" OR "multi-model database"
OR "graph database" OR "edge database" OR "fog database" OR "autonomous database")
AND
("survey" OR "review" OR "tutorial" OR "taxonomy" OR "systematic" OR "state-of-the-art" OR
"overview")
```

4.2. Study selection

The inclusion and exclusion criteria listed in Table 6 were applied in a two-phase process: (1) title and abstract screening, and (2) full-text assessment.

4.3. Quality assessment

Each candidate survey was evaluated against eight quality assessment criteria (QA1–QA8), scored on a 3-point scale (0 = No, 0.5 = Partially, 1 = Yes). Given the breadth of the expanded 1999–2026 identification pool, the inclusion threshold was set at 6 out of 8; papers scoring below this threshold were excluded (papers scoring between 4 and 6 are reported explicitly in the selection flow). QA7 was scored under an explicit venue-tier rule—1 for venues of established

Table 6
Inclusion and exclusion criteria.

ID	Criterion
Inclusion Criteria	
IC1	The paper is a secondary study (survey, review, tutorial, or systematic mapping study) or a cross-system comparative experimental evaluation that synthesises the relative behaviour of multiple systems or techniques; such evaluation studies are admitted but labelled separately (Type E) and treated as empirical rather than secondary evidence
IC2	The papers primary subject is either (i) database-management technology operated as, or directly underpinning, a cloud data service, or (ii) distributed data-management technology (consistency, replication, transactions, partitioning, federated query processing) foundational to cloud database platforms; general cloud-computing, big-data, or application-domain surveys without a database-systems focus do not qualify (cf. EC2)
IC3	The paper was published between January 1999 and the search date (July 2026)
IC4	The paper is written in English
IC5	The paper is peer-reviewed, or is an arXiv preprint that is otherwise subject to the full quality assessment with its venue-quality item (QA7) capped at 0.5
Exclusion Criteria	
EC1	The paper is a primary study (not a survey/review)
EC2	The paper surveys cloud computing in general without database-specific focus
EC3	The paper is a short paper (<6 pages) or workshop position paper
EC4	The paper is a duplicate or substantially overlapping earlier version
EC5	The full text could not be retrieved (no open-access copy and no manually obtainable copy)
EC6	The venue exhibits predatory indicators: self-claimed impact factors, discontinued or absent reputable indexing (Scopus/DOAJ), a watchlisted publisher, or an unverifiable editorial process

Table 7
Quality assessment checklist for surveyed papers.

ID	Quality Assessment Question
QA1	Are the survey objectives clearly stated?
QA2	Is the scope and coverage of the survey well-defined?
QA3	Is the search/selection methodology described or implied?
QA4	Does the survey provide a taxonomy or classification framework?
QA5	Are the reviewed primary studies adequately described?
QA6	Does the survey identify open challenges or future directions?
QA7	Is the survey published in a recognized venue (journal, top conference, or high-citation arXiv)?
QA8	Is the survey sufficiently comprehensive (≥30 references)?

reputation, 0.5 for minor but verifiable outlets (university, society, or currently Scopus-/DOAJ-indexed publishers), and 0 otherwise—and, independently of the QA total, papers from venues meeting the EC6 predatory-indicator test were excluded in a dedicated venue-integrity audit reported in the selection flow.

4.4. Screening validation and data extraction

Because title/abstract screening, full-text assessment, and quality scoring were carried out with large-language-model (LLM) assistance under fixed written rubrics (disclosed in full in the declaration at the end of the paper), we

validated the screening stage quantitatively. A stratified random sample of 200 records—150 drawn from those the primary pass excluded and 50 from those it included—was re-screened independently by a second, different LLM configuration applying the same rubric, blind to the first pass's decisions. Raw agreement was 90.0% (180/200) and Cohen's $\kappa = 0.76$, which corresponds to *substantial* agreement on the Landis–Koch scale. The disagreements were asymmetric—17 of 20 were records the first pass excluded that the second would have advanced, indicating the primary pass was the more conservative—and the full list of disagreements, together with the paired decisions and the agreement computation, is provided in the replication package for independent adjudication. We regard this dual-pass validation, rather than a single opaque automated pass, as the appropriate safeguard for an LLM-assisted tertiary study; it does not substitute for a second independent *human* reviewer, whose absence we record as a limitation (Section 4.6).

For each included survey a structured data-extraction form was completed and archived, recording: bibliographic metadata (authors, venue, year, DOI/arXiv identifier); study type (survey/review vs cross-system experimental evaluation); primary topic and venue type; the lifecycle phases addressed; the eight quality-assessment item scores QA1–QA8; and a coding of the survey against each of the eighteen cross-cutting challenges C1–C18 (absent / mentioned / substantively treated). The per-challenge coding underlies the coverage and temporal-attention analyses in Section 5 and is released as a machine-readable matrix (Data availability).

4.5. Results: study selection and demographics

Figure 5 presents the PRISMA 2020 flow diagram of the selection process, with two identification arms: records identified through the scripted database search, and records identified through other methods (the previous version of this catalog, manually collected reports, and backward snowballing). From 7,675 raw database records, 84 surveys were ultimately included in the meta-survey.

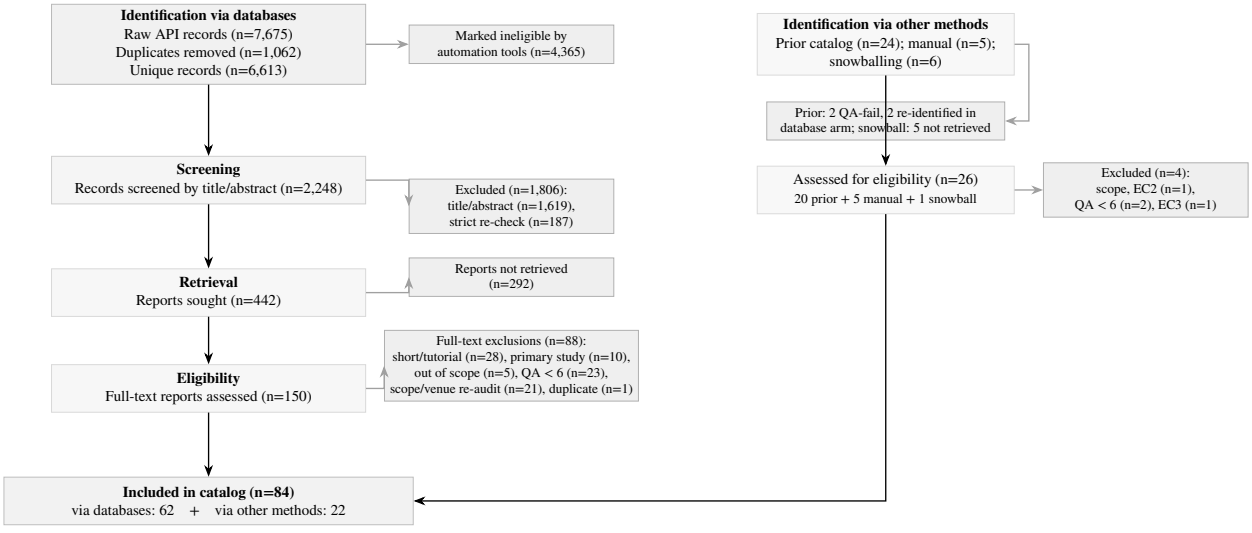

Figure 5: PRISMA 2020 flow diagram of the study selection process [60], with separate identification arms for the scripted database search (left) and other methods (right). All figures are scripted tallies over the archived screening-decision log. In the other-methods arm, of the 24 prior-catalog entries two failed the raised quality threshold and two were re-identified by the database search (and counted there), leaving 20 carried forward; of 5 manually collected reports 3 met all criteria; and of 6 backward-snowballing candidates 5 could not be retrieved and 1 scored below threshold, so none were added.

Figure 6 shows the distribution of selected surveys by publication year. Survey activity is sparse before 2016, grows steadily through the late 2010s, and intensifies markedly from 2024 onward—2024 and 2025 together account for more than a third of the corpus. No 2026 survey met all criteria: the search was run in mid-2026 and indexing, citation accrual, and open-access posting all lag publication, so the most recent months are necessarily under-represented.

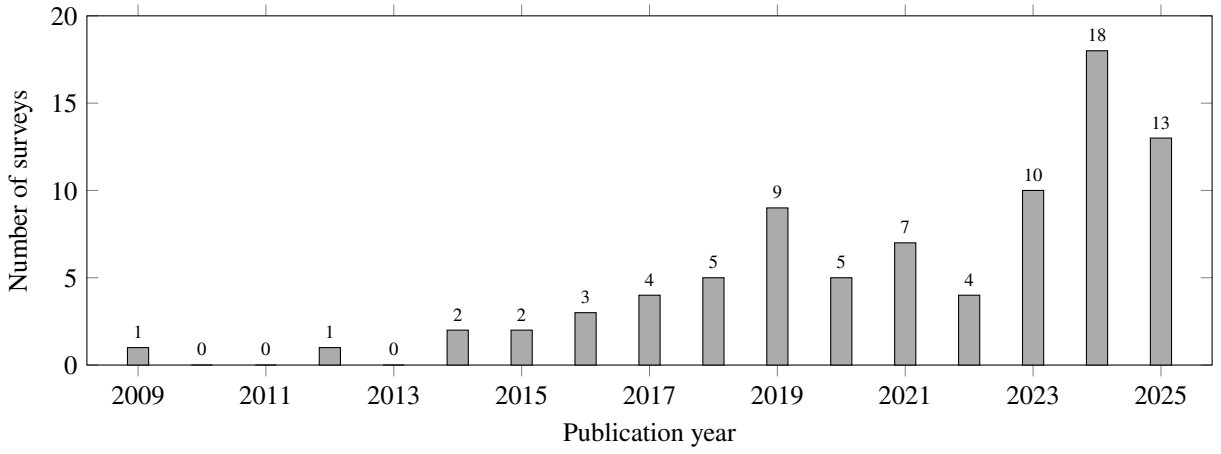

Figure 6: Distribution of the 84 selected surveys by publication year. Although the search window opens in January 1999 and extends to the search date (July 2026), the earliest survey satisfying all inclusion and quality criteria dates from 2009 and the latest from 2025.

Figure 7 shows the distribution by primary topic domain (each survey is assigned a single primary topic, so the counts sum to 84). NoSQL/NewSQL systems are the most frequently surveyed topic, reflecting the maturity and breadth of that body of literature, followed by graph databases and AI4DB/learned database components.

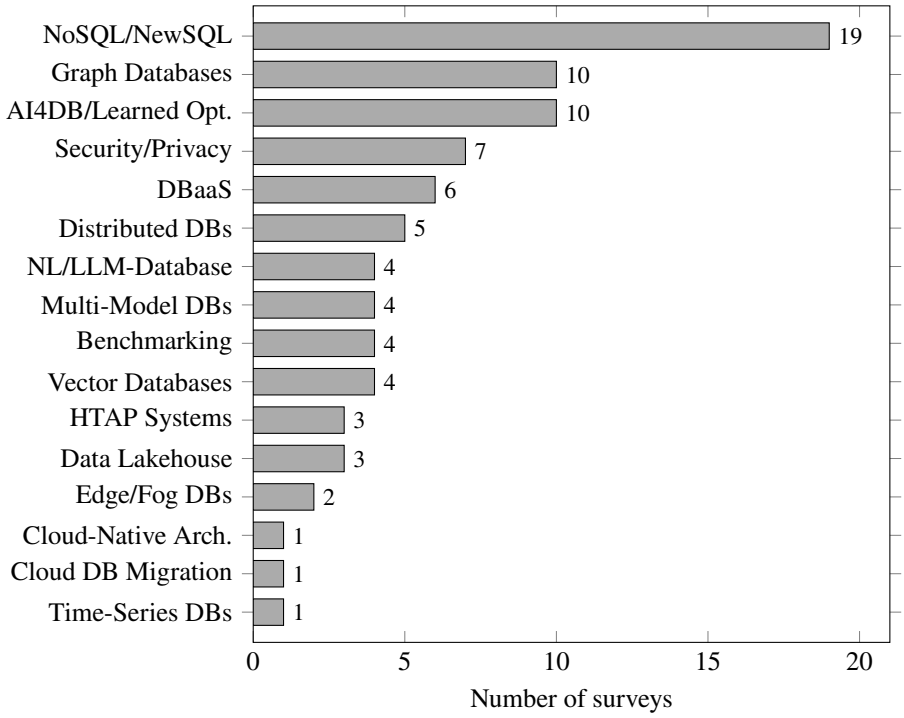

Figure 7: Distribution of the 84 selected surveys by primary topic domain.

Figure 8 shows the distribution by venue type. Journal publications account for the majority, followed by arXiv preprints and conference proceedings; the visible arXiv share reflects both the field's preprint culture and the open-access retrieval policy described in Section 4.6.

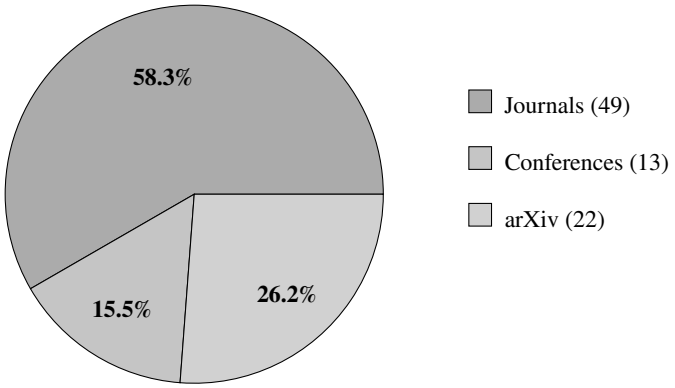

Figure 8: Distribution of the 84 selected surveys by venue type.

4.6. Threats to validity

As with any systematic review, this meta-survey is subject to validity threats, which are mitigated as follows.

Construct validity. The search string and the choice of indexing services may not capture every relevant survey, particularly those using non-standard terminology or absent from the OpenAlex, arXiv, Crossref, and Semantic Scholar indexes; abstract availability in these indexes is also uneven, which affects the automated eligibility marking. These threats are mitigated by (i) running fully scripted queries whose logs are archived and re-executable, (ii) taking the union of four independent APIs rather than relying on a single index, (iii) carrying forward the previous version of the catalog as a separate identification arm, and (iv) a backward-snowballing pass over the reference lists of the highest-scoring included surveys. This pass surfaced six candidate surveys not already in the pool; five could not be retrieved in full text and one scored below the quality threshold, so none entered the catalog. We therefore do not claim search saturation—the snowballing outcome reflects retrieval limits as much as coverage—but the small number of genuinely new candidates found by chasing references is at least consistent with the primary search having captured most of the accessible literature. The English-language filter may exclude relevant surveys in other languages.

Internal validity. Title/abstract screening, full-text assessment, and quality-assessment scoring were performed with the assistance of a large-language-model tool operating under fixed written rubrics (disclosed in the AI-use declaration), with every record's decision and rationale logged; a strict second re-check pass was applied to all provisional inclusions, a dedicated venue-integrity audit re-examined venue quality, and all borderline and final decisions were adjudicated by the author. Every included survey was additionally verified against a resolvable DOI or arXiv identifier and a retained, content-verified full-text copy, so that the catalog contains only confirmable publications. The absence of a second independent *human* reviewer, and the reliance on AI assistance in screening, nonetheless remain limitations.

External validity. As a survey of surveys, the findings inherit the scope and currency limitations of the primary surveys; rapidly evolving topics (e.g., LLM–database convergence) may be underrepresented relative to their current activity. The corpus is further subject to an open-access retrieval bias: every included survey must have a content-verified full text on file, and 292 otherwise-eligible reports could not be retrieved through any open-access channel (reported explicitly in the PRISMA flow), so paywalled surveys—particularly from the early 2000s—are underrepresented. Notably, no survey published before 2009 satisfied all criteria, so the 1999–2008 portion of the window is represented in the narrative (Section 3.2) rather than the catalog. The synthesis is therefore intended as a structured map of the field rather than an exhaustive account of every primary system.

Sensitivity to the quality threshold. The inclusion threshold (QA $\geq$ 6) is a design choice, so we checked how the corpus responds to a stricter bar. Raising it to QA $\geq$ 6.5 removes 14 of the 84 surveys (leaving 70); the topic ranking is unchanged (NoSQL/NewSQL, graph databases, and AI4DB remain the three largest domains) and the 2024–2025 concentration persists (2024 falls from 18 to 15 surveys, 2025 from 13 to 12). The demographic picture and the challenge rankings are therefore stable under a reasonable tightening of the bar; the main effect of the stricter threshold is to thin the minor-venue tail rather than to reshape the field, which is why we report at QA $\geq$ 6 and flag the borderline (QA = 6) entries explicitly in the catalog.

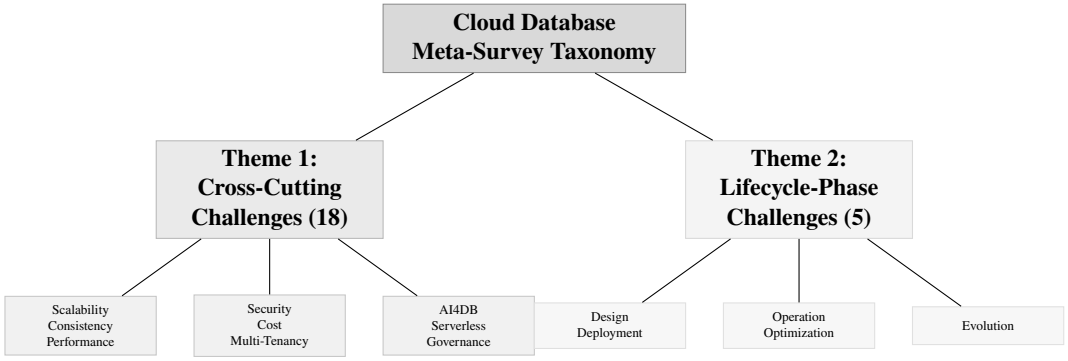

Figure 9: High-level taxonomy of the cloud database meta-survey, organized into two complementary themes.

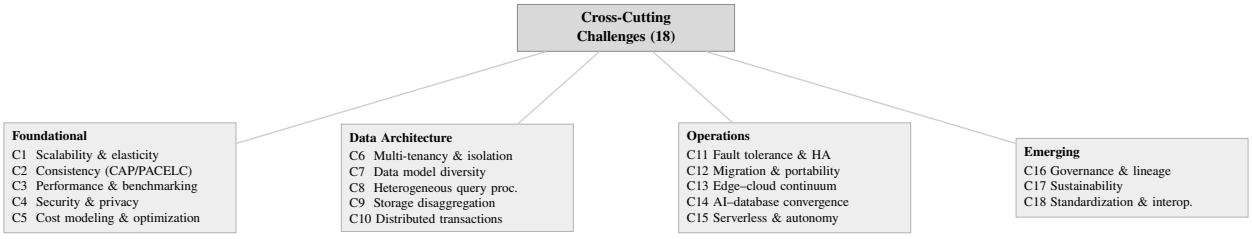

Figure 10: The eighteen cross-cutting challenges, organized into four clusters.

Conclusion validity. The challenge taxonomy and the consensus/divergence synthesis (Table 9) involve interpretive judgment. These are grounded in explicit per-challenge evidence and cross-referenced to the surveyed domains in Table 8 to keep the reasoning traceable.

5. Discussion—Challenges and Research Directions

This section presents the core contribution of this meta-survey: a comprehensive, structured analysis of the challenges and research directions identified across the 84 surveyed papers. Following the dual-theme approach of Saeed and Omlin [59], the discussion is organized into two complementary themes:

1. **Theme 1: Cross-cutting challenges** (Section 5.1)—eighteen recurring challenges that transcend individual system categories.
2. **Theme 2: Lifecycle-phase challenges** (Section 5.2)—challenges organized by the five phases of the cloud database lifecycle.

Figure 9 provides a high-level overview of the dual-theme taxonomy.

5.1. Theme 1: Cross-cutting challenges

Eighteen cross-cutting challenges that recur across multiple cloud database survey domains are identified and organized, in Figure 10, into four clusters—*foundational*, *data-architecture*, *operational*, and *emerging* concerns. Each challenge is discussed as a short narrative that frames the problem, summarizes the current state reported across the surveyed papers (including points of consensus and divergence), and closes with a brief statement of the open problems and research directions—highlighting data- and knowledge-engineering (DKE) relevance—that the surveys leave unresolved.

Three consolidated views support the discussion. Table 8 maps each challenge to the survey domains that address it, revealing coverage density and thematic clusters; Table 9 synthesizes the key points of agreement and divergence across surveyed papers; and Table 10 shows how attention to each challenge has evolved over the study window. Detailed discussion follows in Sections 5.1.1–5.1.18.

Table 8: Cross-cutting challenge coverage across consolidated survey domains. Symbols: • = primary focus, ∘ = substantive discussion, · = mentioned. The rightmost column (*n*) counts domains addressing each challenge.

#	Challenge	Cloud-Native	HTAP	NoSQL / NewSQL	AI4DB / Learned	Vector / Graph / Multi-Model	Lakehouse	Time-Series	Edge / Fog	NL / LLM–DB	Security / Privacy	Benchmarking	Migration	Distributed DBs	DBaaS	*n*
Foundational																
C1	Scalability & elasticity	•	∘	•	·		∘	∘	∘			∘		∘	∘	40
C2	Consistency & CAP/PACELC	•	•	•		·			∘					•		32
C3	Performance & benchmarking	•	∘	∘	•		∘					•		∘		65
C4	Security & privacy	∘		·					∘		•		·		∘	27
C5	Cost modeling & optimization	∘					∘					•	∘		∘	16
Data Architecture																
C6	Multi-tenancy & isolation	•									∘	∘			•	8
C7	Data model diversity		·	•		•	∘	∘								44
C8	Heterogeneous query processing	∘	•	·	∘	∘	•			∘				∘		19
C9	Storage disaggregation	•	∘				•									19
C10	Distributed transactions	•	•	•		·			·					•		27
Operations																
C11	Fault tolerance & HA	•		∘					∘					•		30
C12	Migration & portability	∘		·									•		∘	14
C13	Edge–cloud continuum	·							•		·					7
C14	AI–database convergence	∘			•	·	∘			•						31
C15	Serverless & autonomy	•			∘							·			∘	10
Emerging																
C16	Governance & lineage	·					∘			·	•		·		·	10
C17	Sustainability	·			·							∘				6

Continued on next page

Table 8 continued

#	Challenge	Cloud-Native	HTAP	NoSQL / NewSQL	AI4DB / Learned	Vector / Graph / Multi-Model	Lakehouse	Time-Series	Edge / Fog	NL / LLM–DB	Security / Privacy	Benchmarking	Migration	Distributed DBs	DBaaS	*n*
C18	Standardization & interoperability	∘		∘		•	∘			·			∘	·		19

Table 9: Synthesis of cross-cutting challenges: key similarities (consensus) and differences (divergences) across the 84 surveyed papers. The "survey domains" column reports, for each challenge, the number of individual surveys that substantively address it (per-paper coding matrix, Section 4.4), noting the domains in which they concentrate.

#	Challenge	Survey domains (n)	Key Similarities (Consensus)	Key Differences (Divergences)
C1	Scalability & elasticity	40 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, NoSQL/NewSQL, Lakehouse, Time-Series, Edge/Fog, Benchmarking, Distributed DBs, DBaaS	All surveys agree that disaggregated storage enables elastic scaling; horizontal scale-out is preferred over vertical scaling; auto-scaling is a baseline cloud capability.	Cloud-native surveys focus on component-level elasticity; NoSQL surveys emphasize partition-based data scaling; HTAP surveys and edge/fog surveys stress elasticity under mixed and intermittently connected workloads, respectively.
C2	Consistency & CAP/PACELC	32 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, NoSQL/NewSQL, Edge/Fog, Distributed DBs	CAP/PACELC trade-offs are universally acknowledged; all surveys recognize a spectrum from strong to eventual consistency; configurable consistency is a key cloud differentiator.	NoSQL surveys accept eventual consistency as pragmatic; HTAP surveys insist on strong consistency for transactional correctness; cloud-native surveys present tunable consistency as an application-level choice.
C3	Performance & benchmarking	65 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, NoSQL/NewSQL, AI4DB/Learned, Lakehouse, Benchmarking, Distributed DBs	Cloud-specific factors (network latency, multi-tenancy, cold starts) complicate optimization; existing benchmarks (TPC-C/H, YCSB) are widely considered inadequate for cloud settings.	AI4DB surveys emphasize learned optimizers and indexes; benchmarking surveys focus on measurement methodology and fairness; cloud-native surveys focus on disaggregation overhead.

Continued on next page

Table 9 continued

#	Challenge	Survey domains (n)	Key Similarities (Consensus)	Key Differences (Divergences)
C4	Security & privacy	27 surveys substantively (of 84), concentrated in Cloud-Native, Edge/Fog, Security/Privacy, DBaaS	Encryption at rest and in transit is a baseline requirement; the shared responsibility model is universally acknowledged; GDPR compliance is a common concern.	Security surveys deeply analyze threat models and advanced techniques (homomorphic encryption, confidential computing); cloud-native surveys treat security as one concern among many; edge/fog surveys emphasize privacy-preserving aggregation.
C5	Cost modeling & optimization	16 surveys substantively (of 84), concentrated in Cloud-Native, Lakehouse, Benchmarking, Migration, DBaaS	Cost is a primary driver of cloud adoption; pricing complexity (10–100× variation) is widely noted; pay-per-use models are a defining benefit.	Benchmarking surveys focus on total cost of ownership comparison; lakehouse surveys weigh storage–compute cost trade-offs; migration surveys highlight hidden migration and lock-in costs; cloud-native surveys note multi-dimensional pricing complexity.
C6	Multi-tenancy & isolation	8 surveys substantively (of 84), concentrated in Cloud-Native, Security/Privacy, Benchmarking, DBaaS	Noisy-neighbor performance interference is universally recognized; the trade-off between resource sharing efficiency and isolation is a fundamental tension.	Cloud-native surveys focus on architectural isolation levels (shared-nothing to shared-table); security surveys emphasize tenant data isolation; benchmarking surveys measure interference under controlled conditions.
C7	Data model diversity	44 surveys substantively (of 84), concentrated in NoSQL/NewSQL, Vector/Graph/Multi-Model, Lakehouse, Time-Series	Polyglot persistence is the operational reality; cross-model query support is needed; multi-model databases are gaining traction.	NoSQL surveys favor model-specific optimization; multi-model surveys advocate unified backends; specialized surveys argue that model-native operations (vector, graph, time-series) cannot be efficiently reduced to a single backend.

Continued on next page

Table 9 continued

#	Challenge	Survey domains (n)	Key Similarities (Consensus)	Key Differences (Divergences)
C8	Heterogeneous query processing	19 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, AI4DB/Learned, Vector/Graph/Multi-Model, Lakehouse, NL/LLM–DB, Distributed DBs	Federated query processing across engines is increasingly necessary; predicate pushdown is the baseline optimization technique.	HTAP surveys focus on OLTP–OLAP workload routing; lakehouse surveys emphasize open-format access layers; multi-model surveys and natural-language interface surveys address cross-model and intent-driven query routing.
C9	Storage disaggregation	19 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, Lakehouse, NoSQL/NewSQL	Compute-storage separation is the defining cloud-native architectural principle; write amplification reduction is a key benefit; independent layer scaling is a primary motivation.	Cloud-native surveys focus on log-as-database architectures (Aurora, PolarDB); lakehouse surveys emphasize open table formats; HTAP surveys address separate analytical and transactional storage tiers.
C10	Distributed transactions	27 surveys substantively (of 84), concentrated in Cloud-Native, HTAP, NoSQL/NewSQL, Distributed DBs	2PC and consensus protocols (Paxos/Raft) are the dominant approaches; the consistency–latency trade-off is universally acknowledged.	Cloud-native surveys cite clock synchronization innovations (TrueTime, HLC); NoSQL surveys accept weaker guarantees (CRDTs, eventual consistency); HTAP surveys focus on OLTP–OLAP transaction isolation.
C11	Fault tolerance & HA	30 surveys substantively (of 84), concentrated in Cloud-Native, NoSQL/NewSQL, Edge/Fog, Distributed DBs	Multi-AZ replication is the baseline approach; automated failover is expected; RTO/RPO targets drive architectural decisions.	Cloud-native surveys focus on quorum protocols (e.g., Aurora's 4/6 write quorum); NoSQL surveys emphasize tunable replication factors; edge/fog surveys must handle intermittent connectivity.
C12	Migration & portability	14 surveys substantively (of 84), concentrated in Cloud-Native, Migration, DBaaS	Migration is complex, error-prone, and labor-intensive; schema heterogeneity is the primary technical barrier; vendor lock-in is widely criticized.	Migration surveys focus on methodology and zero-downtime techniques; cloud-native surveys treat migration as a cloud adoption path; NoSQL surveys highlight data model mismatch challenges.

Continued on next page

Table 9 continued

#	Challenge	Survey domains (n)	Key Similarities (Consensus)	Key Differences (Divergences)
C13	Edge–cloud continuum	7 surveys substantively (of 84), concentrated in Edge/Fog	Data placement across edge–fog–cloud tiers requires intelligent policies; intermittent connectivity is the defining constraint.	Edge/fog surveys focus on resource-constrained processing and synchronization; cloud-native surveys view edge as a cloud extension; security surveys emphasize privacy-preserving edge aggregation.
C14	AI–database convergence	31 surveys substantively (of 84), concentrated in Cloud-Native, AI4DB/Learned, Lakehouse, NL/LLM–DB	Both AI4DB and DB4AI directions are growing rapidly; production deployment of learned components remains limited; training data management is a shared challenge.	AI4DB surveys focus on individual learned components (indexes, optimizers); natural-language and retrieval-augmented surveys address LLM-driven querying and retrieval; cloud-native surveys view AI as a system enhancement.
C15	Serverless & autonomy	10 surveys substantively (of 84), concentrated in Cloud-Native, AI4DB/Learned, DBaaS	Serverless abstracts infrastructure management; cold-start latency is a known limitation; pay-per-query suits variable workloads.	Cloud-native surveys treat serverless as one deployment model and catalog autoscaling mechanisms; AI4DB surveys envision full-stack autonomous databases that self-tune beyond the serverless abstraction.
C16	Governance & lineage	10 surveys substantively (of 84), concentrated in Lakehouse, Security/Privacy	End-to-end lineage tracking is increasingly important; regulatory compliance (GDPR, CCPA) drives governance adoption; metadata management is fragmented across tools.	Security surveys focus on access control and audit logging; lakehouse surveys emphasize unified data cataloging; migration surveys track lineage across system transitions; cloud-native and NL-interface surveys note metadata's role in querying.
C17	Sustainability	6 surveys substantively (of 84), concentrated in Benchmarking	Energy-proportional computing is an aspirational goal; carbon-aware data placement is an emerging research direction.	Cloud-native surveys note the energy implications of disaggregation; AI4DB surveys flag the training cost of learned components; benchmarking surveys propose adding energy and carbon metrics; most other survey domains do not yet address sustainability.
C18	Standardization & interoperability	19 surveys substantively (of 84), concentrated in Cloud-Native, NoSQL/NewSQL, Vector/Graph/Multi-Model, Lakehouse, Migration	Vendor lock-in is widely criticized; SQL remains the primary query lingua franca; open formats (Iceberg, Delta Lake, Hudi) are gaining traction.	Graph and specialized surveys highlight query-language fragmentation (Cypher, SPARQL, GQL); NoSQL surveys focus on API heterogeneity; multi-model and natural-language interface surveys push unified access, while migration surveys stress portable formats.

Table 10: Temporal attention to each cross-cutting challenge: the number of surveyed papers, by publication period, that substantively address the challenge (code 2 in the per-paper coding matrix, Section 4.4). Cell shading is scaled within each row; attention concentrates in 2023–2026, mirroring the overall publication trend (Figure 6).

#	Challenge	2009–2015	2016–2019	2020–2022	2023–2024	2025–2026	Total
C1	Scalability & elasticity	3	8	7	15	7	40
C2	Consistency & CAP/PACELC	3	7	7	12	3	32
C3	Performance & benchmarking	4	15	12	21	13	65
C4	Security & privacy	2	7	5	8	5	27
C5	Cost modeling & optimization	1	6	2	4	3	16
C6	Multi-tenancy & isolation	1	3		2	2	8
C7	Data model diversity	3	10	8	18	5	44
C8	Heterogeneous query processing		6	1	10	2	19
C9	Storage disaggregation	1	4	3	7	4	19
C10	Distributed transactions	2	4	5	13	3	27
C11	Fault tolerance & HA	3	9	6	9	3	30
C12	Migration & portability	2	2	1	6	3	14
C13	Edge–cloud continuum		1	2	4		7
C14	AI–database convergence		4	5	14	8	31
C15	Serverless & autonomy		3	2	4	1	10
C16	Governance & lineage		2	2	5	1	10
C17	Sustainability		2	1	3		6
C18	Standardization & interop.	1	7	3	6	2	19
Total (paper-challenge pairs)		**26**	**100**	**72**	**161**	**65**	**424**

5.1.1. Scalability and elasticity

Scalability—handling growing workloads by adding resources—and elasticity—provisioning and releasing resources in response to demand—are the foundational promises of the cloud, and every surveyed cloud-native and NoSQL review treats them as first-order concerns [6, 11]; efforts to quantify elasticity empirically date back to the first wave of cloud data stores [36]. In practice, cloud-native engines such as Aurora [2] and Snowflake [3] achieve storage elasticity through compute–storage disaggregation, while compute elasticity is delivered by auto-scaling and replication [61]; serverless offerings push this further by scaling to zero between requests [29]. On one point the reviews are unanimous: today's elasticity is coarse-grained, acting on whole instances rather than individual components, so *fine-grained* elasticity—scaling a single query operator, the transaction manager, or the log service on its own—is still out of reach [6].

Several obstacles recur across the reviews. Cold-start latency undermines the responsiveness that makes scale-to-zero attractive [29]; state migration during scale-out and scale-in introduces transient performance degradation; and predictive autoscaling depends on workload-forecasting models that are still maturing [62]. Cross-component elasticity in fully disaggregated architectures is the least understood of all, because the interactions among independently scaled layers are difficult to model.

The central open problem is component-level elasticity: scaling a single query operator, the transaction manager, or the log service independently, rather than replicating whole instances. Our reading of the surveys is that this is blocked less by mechanism than by prediction—autoscalers still cannot forecast per-component demand accurately enough to act on it—so progress here depends more on workload modelling than on new architectures.

5.1.2. Consistency, availability, and partition tolerance revisited

The CAP theorem [63] and its PACELC refinement [64] have shaped distributed database design for over a decade, but the surveyed work shows that geo-distribution, multi-region deployment, and the proliferation of configurable consistency levels give these trade-offs new dimensions in the cloud [65]. Cloud databases now expose a spectrum of guarantees: Spanner provides external consistency via TrueTime [48], whereas systems such as Cosmos DB offer several intermediate levels from strong to eventual. Surveys of NoSQL stores [11, 12] and of HTAP systems [7] consistently single out consistency management as a defining difficulty, and they disagree on how much weakening is acceptable—NoSQL reviews accept eventual consistency as pragmatic, while HTAP and NewSQL reviews insist on strong guarantees for transactional correctness.

What the surveys leave open is largely a matter of reasoning and tooling. Developers struggle to predict how a given consistency level affects application correctness [65]; adaptive consistency that adjusts to workload requirements is frequently proposed but rarely deployed; and cross-model consistency—reconciling, say, document and graph views of the same data—is barely studied in multi-model settings [53].

Open problems. The recurring gap is between formal consistency models and everyday development: what is missing is a machine-checkable notion of a consistency *contract*—an explicit statement of the guarantees a service offers and an application requires—so that compatibility can be verified rather than assumed. This is one place where the knowledge-representation toolkit is genuinely the right one, and we return to it in Section 7.

5.1.3. Performance optimization and benchmarking

Performance optimization in the cloud spans query optimization, indexing, caching, resource allocation, and workload management, and benchmarking supplies the empirical ground truth against which systems are compared [66]. The AI4DB and cloud-native reviews document substantial progress on the optimization side—learned query optimization [67], learned indexes [68], and automated knob tuning [69]—yet they emphasize that cloud-specific factors such as network latency, disaggregation overhead, multi-tenant interference, and cold starts confound techniques designed for single-node systems [9, 6]. On the measurement side, the benchmarking reviews are unanimous that classical benchmarks (TPC-C, TPC-H, YCSB) fail to capture cloud realities such as elasticity, pay-per-use pricing, and variable network performance [66, 15, 70, 71].

The unresolved issues follow directly. Cloud-aware benchmarks that incorporate elasticity, multi-tenancy, and pricing are still lacking; fair cross-provider comparison is confounded by differing hardware, pricing, and service-level agreements; and end-to-end performance in disaggregated stacks emerges from complex interactions among independently scaled layers that no current benchmark isolates.

The benchmarking reviews are, in our view, the most unanimous in the whole corpus: every one concludes that TPC-C/H and YCSB no longer measure what matters in the cloud, yet no successor has been agreed. The concrete

need is mundane but unmet—a benchmark that records elasticity, pricing, and multi-tenant interference alongside throughput, and that publishes enough configuration provenance for a result to be reproduced on another provider. Until such a benchmark exists, cross-provider performance claims should be read with scepticism.

5.1.4. Security and privacy

Relative to on-premises systems, cloud databases face an enlarged attack surface arising from shared infrastructure, network-based access, and the need to trust the provider [72, 73, 74]. The security-focused surveys catalog the established defenses—encryption at rest and in transit, access control, audit logging, and vulnerability management [75, 32, 76, 77]—and the more advanced techniques that target computation over protected data, including homomorphic encryption for querying ciphertext [78, 79], architectures giving clients distributed, concurrent access to encrypted cloud databases without a trusted broker [80], private-database models for outsourced data [81], secure multi-party computation, and confidential computing in hardware enclaves. The reviews differ in emphasis: dedicated security surveys analyze threat models in depth, whereas cloud-native reviews treat security as one concern among many and edge-oriented work stresses privacy-preserving aggregation close to the data source.

Across the literature the same hard problems persist: the performance overhead of encryption remains prohibitive for analytics; key management across multi-cloud and hybrid deployments is error-prone; the "honest but curious" provider is difficult to defend against without heavy penalties; and reconciling GDPR's right to erasure with analytic utility is unsolved.

Open problems. Two of the persistent problems—prohibitive encryption overhead for analytics and error-prone multi-cloud key management—are engineering problems that will yield to engineering. The harder one is compliance: reconciling obligations such as GDPR's right to erasure with analytic utility is still done by hand against prose regulations. Making data-sensitivity and policy machine-readable so that compliance can be checked rather than asserted is the direction we find most promising, and the least explored in the surveyed work.

5.1.5. Cost modeling and optimization

Cloud databases bill along many dimensions at once—compute hours, storage, I/O operations, network egress, backups, and premium features such as encryption or multi-region replication—which makes cost both a primary adoption driver and a persistent source of surprise [27]. Empirical studies show that the cost of the same workload can vary by one to two orders of magnitude depending on configuration [27], and although work on serverless analytics [29] and the cloud-native reviews [6] flag cost as critical, they offer little quantitative guidance. The benchmarking and migration reviews add the complementary observations that total-cost-of-ownership comparison and hidden lock-in costs are rarely modeled rigorously [15, 82].

Consequently, pre-execution cost estimation for complex analytical queries remains unreliable; multi-objective optimization over cost, performance, and consistency lacks mature frameworks; the trade-offs among reserved, on-demand, and spot capacity for database workloads are understudied; and vendor lock-in obscures cross-provider comparison.

Cost is the challenge where the gap between practitioner pain and research attention is widest: order-of-magnitude cost swings from configuration choices are common knowledge, yet only four surveyed domains treat cost as a first-order concern (Table 8). What is missing is not another pricing calculator but pre-execution cost *prediction* for non-trivial analytical queries, and a way to reason jointly about cost, performance, and consistency instead of optimising one and hoping for the others.

5.1.6. Multi-tenancy and resource isolation

Multi-tenancy lets providers serve many customers from shared infrastructure, improving utilization and lowering cost, at the price of harder performance isolation, data separation, and fair resource allocation [6]. The surveyed systems realize tenancy at several granularities—shared-nothing (an instance per tenant), shared-process (logical isolation within one engine), and shared-table (row-level security over common tables)—and the reviews repeatedly identify "noisy neighbor" interference, in which one tenant's load degrades another's latency, as the dominant operational hazard [83].

Multi-tenancy is, tellingly, one of the least-surveyed challenges (Table 8), and we suspect this is because the hardest part—strong performance isolation without dedicating hardware—has no clean academic formulation and is solved pragmatically inside proprietary systems. The tractable research question is narrower: predicting one tenant's

interference on another from workload signatures, which would at least let schedulers act before latency degrades rather than after.

5.1.7. Data model diversity and multi-model integration

The cloud has multiplied the data models in routine use—relational, document, graph, key-value, wide-column, vector, and time-series—and applications increasingly need to query across several at once [53, 84, 85, 86]. Multi-model engines such as Cosmos DB, ArangoDB, and SurrealDB expose multiple model interfaces over a single backend, and the NoSQL and multi-model surveys report growing adoption alongside a candid account of the limits: cross-model query optimization and unified transaction semantics remain weak [11, 87, 88, 89, 53]. The reviews also diverge on strategy—NoSQL surveys favor model-specific optimization, multi-model surveys advocate unified backends, and specialized reviews argue that vector, graph, and time-series operations cannot be reduced efficiently to a single engine.

Open problems. The gaps are a query language that spans relational, graph, document, and vector operations; optimisation of queries that mix, say, a graph traversal with a relational join and a similarity search; and schema mapping across models. The surveys divide sharply on whether one engine can serve all models well, and on the evidence we side with the sceptics—native vector, graph, and time-series operations have resisted efficient reduction to a common backend, and a shared conceptual layer over specialised engines looks more realistic than a single unified store.

5.1.8. Query processing across heterogeneous engines

Modern cloud data stacks rarely rely on a single engine; OLTP, OLAP, graph, search, and vector systems must be queried in concert, making federated query processing and data virtualization essential [7, 90]. Engines such as Presto/Trino, BigQuery Omni, and AWS Lake Formation already provide federation, and the HTAP reviews analyze how queries should be routed to the appropriate engine [7]; yet they agree that cross-engine optimization is still rudimentary, seldom going beyond predicate pushdown. The unresolved problems are cost-based optimization across engines with incompatible cost models and capabilities, preserving transactional guarantees across engine boundaries, and the semantic understanding of data needed to route queries intelligently rather than syntactically.

The concrete missing piece is cost-based optimisation *across* engines with incompatible cost models—today's federation seldom advances beyond predicate pushdown. Ontology-mediated query answering [25], which poses one query against many engines through a shared conceptual schema, is the most developed idea here and connects the problem to a long line of data-integration research.

5.1.9. Storage disaggregation and log-as-database

Separating compute from storage into independently scalable layers is the defining architectural move of the cloud-native era, and the "log-as-database" pattern pioneered by Aurora—treating the redo log itself as the system of record—pushes intelligence into the storage tier [2, 6]. This design reduced write amplification and enabled fast failover, and systems such as PolarDB [19] and Socrates [4] carry disaggregation further by splitting out additional components. The cloud-native surveys report that while disaggregation simplifies operations and scaling, it introduces its own difficulties in remote-I/O latency, cache coherence, and log management [6].

Where the effort should go. The open issues are concrete and mechanical: hiding the latency gap between remote storage and local SSDs, keeping caches coherent across layers, and managing a log that never stops growing. The deeper question the surveys raise but do not answer is a placement one—given a disaggregated stack, where should each operator actually run—and we expect the next round of cloud-native engines to compete precisely on how well they answer it.

5.1.10. Transaction processing in distributed cloud environments

Distributed transaction processing must balance ACID guarantees against performance and scalability, and the surveyed systems span the full design space from strongly consistent protocols (two-phase commit, Paxos/Raft) to relaxed models such as CRDTs and eventual consistency [48, 18]. Spanner demonstrated that globally distributed, strongly consistent transactions are feasible using synchronized clocks [48], and CockroachDB and YugabyteDB provide comparable guarantees with hybrid logical clocks [18]; the NewSQL and HTAP reviews nonetheless report that sustaining strong consistency at high throughput for mixed workloads is still hard [7].

Open problems. Three problems recur: cutting commit latency for geo-distributed transactions without weakening consistency, resolving cross-region conflicts efficiently, and isolating transactional from analytical work inside a single

HTAP engine so neither starves the other. Spanner settled the feasibility question a decade ago; what the surveys show is that doing the same at commodity latency and cost has not been settled.

5.1.11. Fault tolerance, recovery, and high availability

Cloud databases must survive hardware faults, network partitions, availability-zone outages, and even whole-region failures while preserving durability and minimizing downtime [2, 6]. The prevailing approach is multi-availability-zone replication—typically three-way—with automated failover, exemplified by Aurora's six-replica, three-zone write quorum [2]. The surveys report that recovery-time objectives of seconds and near-zero recovery-point objectives are attainable within a single region, but remain difficult to guarantee across regions.

Single-region resilience is essentially solved; the open frontier is multi-region recovery with near-zero data loss, and consistent recovery when state is scattered across a disaggregated stack. One gap struck us as surprisingly basic: the surveys note the near-absence of chaos-engineering and fault-injection frameworks built for databases specifically, so resilience is still argued anecdotally rather than measured.

5.1.12. Data migration and cross-cloud portability

Organizations routinely move databases between on-premises and cloud, between providers, and between services, and the migration survey documents how hard it is to preserve consistency, minimize downtime, and transform schemas along the way [82]. Although every major provider offers a migration service, the reviewed evidence is that migrations remain error-prone and labor-intensive, with schema and data-model mismatches the principal technical barrier and vendor lock-in—through proprietary APIs, formats, and features—compounding the difficulty of any subsequent move [91].

Open problems. Three needs recur: zero-downtime migration for large, continuously written databases; automated schema mapping between heterogeneous systems; and portability layers that abstract away provider-specific features. Migration is also the single least-covered challenge in the corpus and maps onto the least-served lifecycle phase (evolution), which we read as a genuine blind spot rather than a solved problem—the tooling exists, but the surveyed evidence is that it remains manual and error-prone.

5.1.13. Edge–cloud data continuum

The spread of IoT devices, autonomous vehicles, and smart-city infrastructure has created demand for database capability at the network edge operating in concert with the cloud [13], a pressure recognized early in evaluations of cloud databases for IoT data [92]. The edge-and-fog survey maps architectures that distribute storage and query processing across edge devices, fog nodes, and cloud backends, and notes that systems such as Azure SQL Edge, AWS Outposts, and locality-aware features in CockroachDB address only parts of the problem [13].

Research directions. The unresolved problems are consistency across the edge–fog–cloud tiers under intermittent connectivity, deciding what to cache or move where, querying under tight edge resource budgets, and aggregating privately before data reaches the cloud. The unifying need is a placement policy that reasons jointly about latency, privacy, and connectivity—today these are traded off by hand, per deployment.

5.1.14. AI–database convergence (AI4DB and DB4AI)

The convergence of AI and databases runs in two directions—AI4DB, which uses learning to optimize the database, and DB4AI, which uses database technology to serve AI workloads [9]. The AI4DB surveys document concrete progress in learned query optimization [67], learned indexes [68, 93, 94], automated knob and buffer tuning [69, 95], learned partitioning advisors [96], cardinality estimation, and workload forecasting [62, 97], while DB4AI spans in-database machine learning and optimized data serving for training pipelines. The reviews agree that learned components are rarely yet trusted in production, and that managing their training data is a shared, unsolved concern. The persistent obstacles are deploying learned components with safety guarantees so that they degrade gracefully when a model fails, collecting and curating training data from live workloads, transferring learned behavior across instances and hardware, and—perhaps most important for adoption—explaining why a learned optimizer made a given decision.

Open problems. The blocker to adoption is not accuracy but trust: the surveys agree that learned optimisers and indexes rarely run unsupervised in production because they fail unpredictably and cannot explain their choices. In our view the most valuable near-term work is therefore not a better learned component but a safety envelope—mechanisms that let a learned optimiser degrade gracefully to a known-good plan when its confidence is low.

5.1.15. Serverless and autonomous database operations

Serverless databases hide infrastructure entirely, provisioning, scaling, patching, and tuning without operator involvement, and autonomous databases go further by using machine learning to self-diagnose and self-heal [47, 29]. Offerings such as Aurora Serverless v2, the Cosmos DB serverless tier, Neon, and PlanetScale realize the serverless interface, while Oracle Autonomous Database is the most ambitious autonomy effort; studies of serverless data systems report that the model suits variable workloads well but struggles with sustained high throughput and cold-start latency [29].

The open problems are predictable performance under autoscaling, an honest exposure of the cost–performance trade-off to the user, and full-stack autonomy that unifies self-tuning, self-healing, and self-securing. The surveys converge on one point worth stating plainly: genuine autonomy needs a database that can reason about its own behaviour rather than react to it, and no surveyed system yet does this across the full stack.

5.1.16. Data governance, provenance, and lineage

As cloud data ecosystems grow, tracking where data came from (provenance), how it was transformed (lineage), and whether its use complies with policy becomes essential for compliance, debugging, and trust [26]. The governance-related surveys identify a common set of needs—metadata management, data cataloging, access control, quality monitoring, and regulatory compliance—and report that cloud-native tools such as AWS Glue Data Catalog, Azure Purview, and Google Dataplex address them only partially, leaving the landscape fragmented and unstandardized. The unresolved problems are end-to-end lineage that spans heterogeneous services and processing engines, automated checking against regulatory requirements that themselves keep changing, and provenance-aware query processing that records how each result was derived.

Research directions. Governance is the challenge where the knowledge-representation angle is, in our judgement, most clearly the right one: representing metadata, lineage, and policy as a linked knowledge graph over cloud data assets would let cataloguing, lineage, and compliance be queried and reasoned over together, instead of through the fragmented per-tool catalogs that the surveys describe today.

5.1.17. Sustainability and energy efficiency

Data centres consumed, per the IEA's assessment, on the order of 1–1.5% of global electricity in recent years, with projections rising toward 3–4% by 2030 as AI and cloud workloads grow, and cloud databases—heavy consumers of compute and storage—contribute directly to that footprint [98]. Work on green computing identifies opportunities in energy-efficient query processing, scheduling that follows renewable-energy availability, and carbon-aware data placement; major providers now report carbon metrics, but the surveyed evidence is that database-level energy optimization is still nascent.

Sustainability is the least-addressed challenge in the entire corpus—only six surveys treat it substantively (Table 10)—which we read less as a judgement on its importance than as a measure of how new it is to the database community. The obvious directions are energy-proportional databases, carbon-aware placement that follows cleaner grids, and benchmarks that report joules and carbon next to throughput; that none of these is yet standard practice is itself the finding.

5.1.18. Standardization and interoperability

The proliferation of cloud database systems, query languages, APIs, and formats has produced an interoperability problem: SQL remains the lingua franca for relational data, but graph databases alone span Cypher, SPARQL, Gremlin, and GQL, and NoSQL systems expose largely proprietary APIs [8]. The surveys note genuine progress—GQL's emergence as an ISO standard, the SQL/PGQ extension for property graphs, and the consolidation of open table formats such as Iceberg, Delta Lake, and Hudi—while cautioning that vendors continue to differentiate through proprietary extensions, keeping true interoperability out of reach.

The recurring asks are a universal query interface across models, standardised control-plane APIs, and open formats that blunt lock-in. The surveys are guardedly optimistic—GQL becoming an ISO standard and the consolidation around Iceberg/Delta/Hudi are real progress—but we would temper that optimism: standards constrain the commodity layer while vendors keep differentiating above it, so interoperability tends to arrive exactly where it no longer confers advantage.

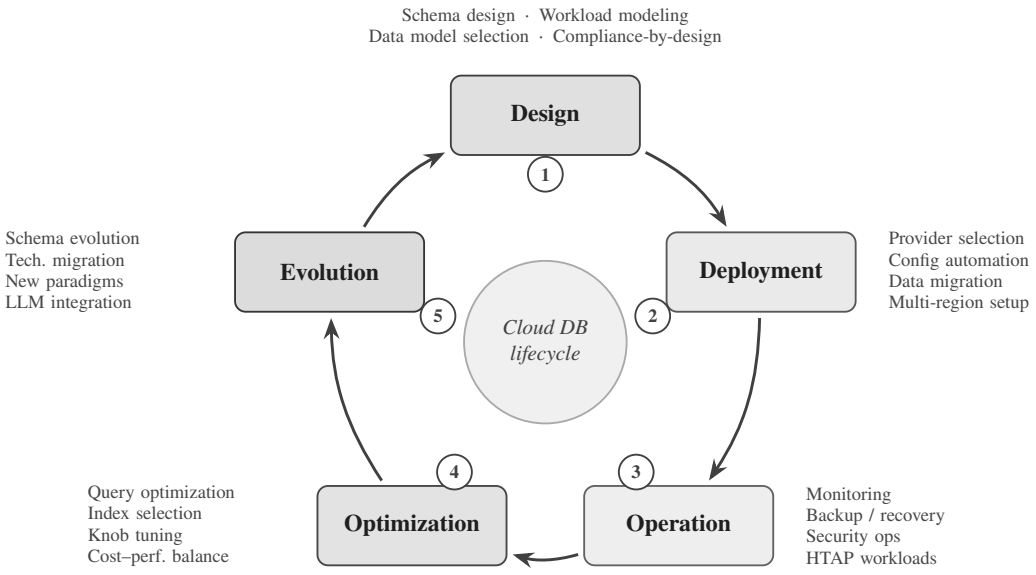

Figure 11: The cloud database lifecycle with key challenges mapped to each phase.

Table 11
Surveyed papers grouped by the lifecycle phase(s) they address (derived from the catalog in Table 12; a survey may span several phases, so the counts sum to more than 84). Representative examples are cited by S-ID; the complete per-phase assignment is part of the archived catalog data.

Phase	Representative challenges	Examples	*n*
Design	Schema design, data-model selection, workload characterization, compliance-by-design	S5, S11, S47, S64, S82	54
Deployment	Provider selection & lock-in, configuration automation, data ingestion/migration, multi-region setup	S4, S9, S18, S57, S84	22
Operation	Monitoring & observability, backup/recovery, security operations, HTAP workload management	S1, S5, S13, S40, S72	39
Optimization	Learned query optimization, index/materialized-view selection, knob tuning, cost–performance balance	S2, S21, S32, S54, S78	65
Evolution	Schema evolution & online DDL, technology migration, new-paradigm adoption, LLM integration	S4, S9, S34, S44, S60	6

5.2. Theme 2: Challenges by cloud database lifecycle phases

Complementing the cross-cutting perspective, challenges are organized by the five phases of the cloud database lifecycle: design, deployment, operation, optimization, and evolution. Figure 11 illustrates this lifecycle with key challenges mapped to each phase, and Table 11 groups the surveyed papers by the phase(s) they address. The distribution is itself revealing: design and optimization are the most heavily surveyed phases, whereas the *evolution* phase—schema change, technology migration, and the adoption of new paradigms such as vector and lakehouse systems—is addressed by the fewest surveys, marking it as the least mature stage of the lifecycle and a clear opportunity for future work.

5.2.1. Design phase

The design phase encompasses decisions made before a cloud database is deployed, including schema design, data model selection, workload characterization, and compliance architecture.

Schema design for cloud-native environments. Traditional schema design principles (normalization, ER modeling) must be adapted for cloud databases where denormalization may improve performance due to the cost of distributed joins [6]. Document and wide-column stores require fundamentally different schema design approaches centered on access patterns rather than entity relationships [11, 99, 100, 101].

Workload characterization. Accurate workload modeling is essential for selecting the right cloud database service, configuring resources, and estimating costs. Surveys on AI4DB [9] discuss workload forecasting using ML, but applying these techniques during the design phase—before production data exists—remains challenging.

Data model selection. Choosing between relational, document, graph, key-value, wide-column, vector, or multi-model databases requires understanding workload characteristics, query patterns, consistency requirements, and cost constraints. No systematic decision framework exists that accounts for all these dimensions [53].

Compliance-by-design. Incorporating regulatory requirements (GDPR, HIPAA, data sovereignty) into the initial database design rather than retrofitting compliance is increasingly recognized as essential [31]. This includes data residency constraints, encryption requirements, and audit trail design.

5.2.2. Deployment phase

The deployment phase covers provider selection, infrastructure configuration, data ingestion, and multi-region setup.

Provider selection and vendor lock-in. Choosing a cloud database provider involves evaluating technical capabilities, pricing, compliance certifications, geographic coverage, and ecosystem integration. Vendor lock-in—through proprietary features, APIs, and data formats—is consistently identified as a top concern across surveys [91, 6].

Configuration automation. Infrastructure-as-Code (IaC) tools (Terraform, Pulumi, CloudFormation) enable automated database provisioning, but optimal configuration selection—instance types, storage tiers, replication settings, parameter tuning—often requires deep expertise that is difficult to automate [69].

Data ingestion and migration. Initial data loading and ongoing ingestion from diverse sources (operational databases, streaming platforms, file systems) into cloud databases involves ETL/ELT pipeline design, schema mapping, and data quality assurance [82].

Multi-region deployment. Deploying databases across multiple geographic regions for latency, availability, and compliance reasons introduces challenges in data synchronization, conflict resolution, and region-aware routing [48].

5.2.3. Operation phase

The operation phase covers the day-to-day management of running cloud database systems.

Monitoring and observability. Cloud databases generate vast amounts of telemetry data (metrics, logs, traces). Effective monitoring requires understanding normal behavior patterns, detecting anomalies, and correlating events across distributed components [62]. Work on self-driving databases [47] emphasizes the need for ML-driven anomaly detection and root cause analysis, and production experience with diagnosing intermittent slow queries at cloud scale shows how difficult root-cause attribution remains [102].

Backup, recovery, and disaster recovery. While cloud providers automate basic backups (point-in-time recovery, snapshots), disaster recovery across regions, testing recovery procedures, and meeting stringent RPO/RTO targets remain operational challenges [2].

Security operations. Ongoing security management includes access control auditing, encryption key rotation, vulnerability patching, network security configuration, and compliance monitoring. The dynamic nature of cloud environments (auto-scaling, serverless) complicates security posture management [75].

Real-time workload management (HTAP). For HTAP systems [7], operational challenges include managing resource contention between transactional and analytical workloads, maintaining freshness guarantees for analytical queries, and preventing analytical queries from degrading OLTP performance.

5.2.4. Optimization phase

The optimization phase encompasses techniques for improving performance and efficiency of running cloud databases.

Learned query optimization. ML-based query optimizers [67, 9] that learn from workload history to generate better query plans represent a paradigm shift from traditional cost-based optimization. However, deploying learned optimizers in production requires addressing safety (avoiding catastrophic regressions), training efficiency, and adaptability to workload changes.

Automated index and materialized view selection. Cloud databases increasingly offer automated index recommendation (e.g., Azure SQL automatic tuning, Amazon Redshift automatic materialized views), but surveys [9] report that these systems are often conservative, covering only a fraction of possible optimizations.

Knob tuning. Database configuration parameters (buffer pool size, parallelism degree, checkpoint intervals, etc.) significantly impact performance. ML-based tuning systems [69, 95] can explore the high-dimensional configuration space more effectively than manual tuning, but transferring tuning knowledge across different workloads and hardware remains an open problem.

Cost–performance trade-off optimization. In cloud environments, performance optimization cannot be divorced from cost. The optimal configuration minimizes cost subject to performance constraints (or maximizes performance subject to budget constraints), requiring multi-objective optimization [27].

5.2.5. Evolution phase

The evolution phase addresses long-term changes in database systems, technologies, and requirements.

Schema evolution and online DDL. Changing database schemas in production without downtime (online DDL) is a critical capability for agile development. Cloud-native databases have improved online DDL support, but complex migrations (column type changes, table splits, data backfills) remain challenging [6].

Technology migration. Migrating from one database technology to another (e.g., from a relational database to a document store, or from a legacy system to a cloud-native database) is a major undertaking that surveys identify as poorly supported by existing tools [82].

Adapting to new paradigms. The emergence of vector databases for AI workloads [5], the lakehouse paradigm [17], and serverless computing [29] requires existing systems to evolve or be replaced. Understanding when and how to adopt new paradigms is a strategic challenge.

LLM integration. The rapid rise of large language models (LLMs) is creating new requirements for cloud databases, from vector storage and retrieval for RAG pipelines to natural language interfaces for database querying and administration (discussed in detail in Section 6).

6. Cloud Databases and Large Language Models—A Convergence Case Study

The convergence of cloud databases and large language models (LLMs) represents one of the most dynamic and rapidly evolving areas in modern data management. Unlike the rest of this paper, this section deliberately steps outside the systematic corpus: because the convergence is younger than most of the surveyed literature, only four included surveys (S19, S21, S57, S76) address it directly, so here we draw on individual primary systems to illustrate where the field is heading, anchoring the discussion to those corpus surveys where possible. This mirrors Saeed and Omlin's [59] treatment of XAI in medicine as a domain-specific application, and the reader should read it as an informed outlook rather than a systematic synthesis.

6.1. LLM-augmented database interfaces

Natural language interfaces to databases (NLIDB) have been a long-standing research goal [103]. The advent of LLMs has dramatically improved text-to-SQL accuracy, with systems like DIN-SQL [104], DAIL-SQL [105], and MAC-SQL [106] achieving over 85% execution accuracy on the Spider benchmark [107].

Current capabilities. LLM-based text-to-SQL systems leverage schema-aware prompting, few-shot in-context learning, and chain-of-thought reasoning to translate natural language queries into SQL. Cloud vendors are integrating these capabilities: Amazon Q for databases, Azure AI integration with SQL, and Google Duet AI for BigQuery.

Open challenges.

- Complex queries involving multiple joins, subqueries, and window functions remain error-prone.
- Schema understanding for large databases with hundreds of tables and complex relationships.
- Multi-turn conversational interfaces that maintain context across query refinements.

- Handling ambiguity, domain-specific terminology, and user intent clarification.
- Security risks from SQL injection through natural language prompts.

6.2. Vector databases as LLM infrastructure

Vector databases have emerged as critical infrastructure for LLM applications, particularly retrieval-augmented generation (RAG) pipelines [5].

Architecture and workloads. RAG pipelines embed documents into high-dimensional vectors, store them in vector databases, and retrieve relevant context for LLM prompts based on semantic similarity. This creates unique workload patterns: bulk embedding ingestion, high-throughput approximate nearest neighbor (ANN) search, and hybrid queries combining vector similarity with metadata filtering.

System landscape. The vector database market has rapidly diversified, including purpose-built systems (Pinecone, Weaviate, Milvus, Qdrant, Chroma), vector extensions to existing databases (pgvector for PostgreSQL, Elasticsearch vector search), and cloud-native offerings (Amazon OpenSearch vector engine, Azure AI Search, Google Vertex AI Vector Search) [5].

Open challenges.

- Scalability and freshness—keeping vector indexes up-to-date as source documents change.
- Multi-modal vector search combining text, image, and structured metadata.
- Quality metrics for retrieval that correlate with downstream LLM task performance.
- Cost optimization for storing and querying billions of high-dimensional vectors.
- Integration with existing cloud database ecosystems (transactions, access control, governance).

6.3. LLMs for database administration

LLMs are increasingly applied to automate database administration tasks, including performance diagnosis, query optimization, and anomaly resolution.

Systems and approaches. D-Bot [108] uses LLMs for database diagnosis, performing root cause analysis of performance anomalies by querying system views and analyzing metrics. DB-GPT [109] provides an LLM-powered framework for database interaction, supporting text-to-SQL, data analysis, and administration tasks. GPTuner [110] leverages LLMs for database knob tuning, combining GPT-4 knowledge with Bayesian optimization.

Open challenges.

- Reliability—LLMs can hallucinate incorrect diagnostic conclusions or generate harmful administration commands.
- Safety guardrails—preventing LLM-driven administration from executing destructive operations.
- Integrating LLM reasoning with structured database knowledge (system catalogs, performance models).
- Evaluation frameworks for LLM-based DBA tools that measure real-world effectiveness.

6.4. Challenges and open problems at the convergence

The LLM–database convergence creates several overarching challenges:

1. **Infrastructure co-design.** Jointly optimizing cloud database infrastructure for both traditional workloads and LLM-related workloads (embedding generation, vector search, context retrieval) requires new architectural thinking.
2. **Data governance for AI.** Ensuring that LLM applications respect data access policies, privacy regulations, and audit requirements when retrieving context from cloud databases.
3. **Evaluation and benchmarking.** Standardized benchmarks for vector database performance, text-to-SQL accuracy on real-world schemas, and LLM-based DBA effectiveness are needed.
4. **Cost management.** LLM API calls combined with cloud database queries create complex cost structures that are difficult to predict and optimize.

6.5. Open questions at the cloud-database/LLM frontier

Looking across the challenges catalogued in Section 5, the cloud-database–LLM convergence raises a set of questions that the surveyed literature has only begun to address. They are offered not as a checklist but as prompts for future work in this domain—and, by analogy, for other data-intensive domains:

- How should a cloud database co-schedule traditional query workloads and LLM-driven workloads (embedding generation, vector search, context retrieval) on shared, elastic infrastructure without one starving the other?
- When an LLM retrieves context from governed cloud data, how can access policies, privacy regulations, and audit obligations be enforced *through* the retrieval path rather than around it?
- What would a faithful benchmark for this convergence measure—vector-search quality, text-to-SQL accuracy on real enterprise schemas, or the end-to-end utility of an LLM-assisted DBA—and who maintains its ground truth?
- Can the reliability of LLM-based administration be bounded well enough to let it act on production systems, and what guardrails make graceful degradation the default when the model is wrong?
- How should the combined cost of LLM inference and cloud-database execution be modeled and optimized as a single objective, given that today they are billed and tuned independently?
- What role can knowledge graphs and ontologies play in grounding LLM reasoning in the actual schema, constraints, and semantics of a cloud database—turning a plausible answer into a correct one?

These questions show how the cross-cutting and lifecycle challenges of Section 5 resurface, sharpened, when cloud databases meet LLMs; the same analysis could be re-instantiated for other domains such as scientific data management or real-time analytics.

7. Conclusions

This paper has presented, to the best of our knowledge, the first systematic tertiary study of cloud database research, synthesising findings from 84 content-verified survey papers identified through a reproducible, scripted search covering January 1999 to July 2026 (with the included literature itself beginning in 2009). Applying the Kitchenham–Charters protocol with PRISMA 2020 reporting, the scattered landscape of cloud database challenges has been consolidated into a structured, dual-theme taxonomy.

Answers to the research questions

RQ1 (landscape). The 84 included surveys span 2009–2025 and sixteen topic domains, dominated by NoSQL/NewSQL (19), graph databases (10), and AI4DB/learned optimisation (10); publication activity is concentrated in 2023–2025 (Figures 6–7). Journals account for 58% of venues, arXiv 26%, and conferences 16% (Figure 8); the visible preprint share reflects both the field's culture and our open-access retrieval policy. Eleven of the 84 are cross-system experimental evaluations rather than secondary studies (Type E in Table 12).

RQ2 (cross-cutting challenges). Eighteen challenges recur across domains (Section 5.1); per-paper coding (Table 8) shows that scalability, performance/benchmarking, data-model diversity, and standardisation are the most broadly addressed, while sustainability and the edge–cloud continuum are the least. The challenges are not independent: consistency, transactions, and fault tolerance form one tightly coupled cluster, and storage disaggregation, query federation, and cost form another.

RQ3 (lifecycle mapping). Mapping challenges to the design–deployment–operation–optimization–evolution lifecycle (Table 11) shows design and optimization are the most thoroughly surveyed phases and evolution the least, making schema change, technology migration, and paradigm adoption the most under-served phase.

RQ4 (AI/LLM convergence). Section 6 finds the convergence moving quickly from research to practice through vector databases, text-to-SQL, and LLM-based administration, but with reliability, governance, and evaluation unresolved; only four of the 84 surveys are dedicated to NL/LLM–database topics, so the corpus lags the primary literature here.

RQ5 (open directions). The most widely recurring open problems—fine-grained elasticity, machine-checkable consistency contracts, cloud-aware benchmarking, principled cost models, and trustworthy learned components—are distilled in Section 7 and cut across the majority of surveyed domains.

For the first theme, eighteen cross-cutting challenges are identified and synthesized: (1) scalability and elasticity; (2) consistency and the CAP/PACELC trade-offs; (3) performance optimization and benchmarking; (4) security and privacy; (5) cost modeling and optimization; (6) multi-tenancy and resource isolation; (7) data-model diversity and multi-model integration; (8) heterogeneous query processing; (9) storage disaggregation and the log-as-database pattern; (10) distributed transaction processing; (11) fault tolerance, recovery, and high availability; (12) data migration and cross-cloud portability; (13) the edge–cloud data continuum; (14) AI–database convergence; (15) serverless and autonomous operation; (16) data governance, provenance, and lineage; (17) sustainability and energy efficiency; and (18) standardization and interoperability. Across these, *scalability*, *consistency*, *security*, *cost*, and *AI–database convergence* recur in the majority of the surveyed domains (Table 10), indicating where progress would benefit the field most broadly.

For the second theme, the challenges are mapped to the five phases of the cloud database lifecycle—design, deployment, operation, optimization, and evolution. Design and optimization are the most thoroughly surveyed phases, whereas evolution (schema change, technology migration, and new-paradigm adoption) is the least mature and most under-served (Table 11), making it a particularly promising target for future work.

Two observations cut across both themes. The convergence of cloud databases with large language models is moving rapidly from research to practice through vector databases, text-to-SQL, and LLM-based administration, yet reliability, governance, and evaluation remain unresolved (Section 6). And from a knowledge-engineering standpoint, cloud databases offer rich openings for DKE techniques—knowledge graphs for metadata and lineage, ontology-mediated integration and query answering, and formal models for consistency, cost, and schema evolution.

Accordingly, this meta-survey is intended to serve three purposes: (1) as a comprehensive reference for researchers entering the cloud database space, providing a structured map of the territory and its open problems; (2) as a strategic guide for practitioners navigating technology selection, deployment, and evolution; and (3) as a bridge between the cloud database and the data-and-knowledge-engineering communities.

Limitations. Three limitations qualify these conclusions. First, the corpus is deliberately restricted to surveys that could be content-verified against a resolvable DOI or arXiv identifier, trading breadth for confirmability, so some relevant but harder-to-verify or paywalled surveys may be absent. Second, study selection, quality assessment, and data extraction were LLM-assisted under fixed rubrics and validated by a dual-pass agreement study ($\kappa = 0.76$) with author adjudication, but without a second independent human reviewer (Sections 4.4 and 4.6). Third, as a survey of surveys, the findings inherit the currency limitations of the primary surveys, so the fastest-moving topics—most notably LLM–database convergence—may be under-represented relative to their present activity.

Future work

Based on the analysis of 84 surveys and the challenges discussed in Sections 5 and 6, several priority research directions emerge:

- **Foundational systems.** Cloud-native benchmarks that capture elasticity, multi-tenancy, and pricing; the transition of learned optimizers, learned indexes, and ML-based tuning from prototypes to production-ready, safely-degrading components; and a maturing vector-database ecosystem with standardized APIs and retrieval-quality metrics.
- **Architecture and query processing.** Cross-cloud middleware and portable open formats that reduce vendor lock-in; and federated query processing with consistent transactions across heterogeneous engines and the edge–fog–cloud continuum.
- **Intelligence and autonomy.** Autonomous databases whose learned decisions are explainable and grounded in database knowledge; and intent-driven natural-language interfaces that let users express goals directly while preserving correctness and access control.
- **Governance and sustainability.** Privacy-preserving analytics and end-to-end lineage that scale to regulatory needs; and energy- and carbon-aware database operation that converges the relational, graph, document, vector, and streaming models toward unified, sustainable platforms.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used generative AI large-language-model assistants from the Claude family (Anthropic, 2026 releases): (i) to help draft, edit, and format portions of the manuscript and to prepare its figures and tables; and (ii) to assist in executing the systematic review pipeline—running the scripted literature search, applying the written screening rubrics at the title/abstract and full-text stages, proposing quality-assessment scores, and coding each included survey against the challenge taxonomy—with every machine-assisted decision logged in the archived records (see the Data availability statement). Two distinct model configurations were used for the primary and validation screening passes, and their agreement was measured ($\kappa = 0.76$; Section 4.4). After using these tools, the author reviewed and edited the content, re-checked all provisional inclusion decisions, adjudicated every borderline case and the venue-integrity audit, and takes full responsibility for the content of the publication.

Data availability

This study analyzes previously published survey papers, all of which are cited in the manuscript. The complete identification and selection pipeline is archived with the manuscript sources: the search scripts (`search/search_surveys.py` and companions), the per-query search log (`search_log.json`), the deduplicated candidate pool (`candidates.csv`), the per-record screening-decision log covering every stage of the PRISMA flow (`screening_decisions.csv`), the quality-assessment scores (`qa_scores.csv`), and the frozen machine-readable catalog (`catalog.csv`) from which every figure and table in Section 4.5 is generated. This permits verbatim re-execution of the search and independent audit of every inclusion decision.

A. Complete List of Surveyed Papers with Classification

Table 12 presents the complete catalog of the 84 surveyed papers, classified by study type (survey/review vs cross-system experimental evaluation), primary topic, year, venue, quality-assessment score, and lifecycle phases covered.

Table 12: Master catalog of surveyed papers (S1–S84), ordered chronologically. Type: S = survey/review/SLR, E = cross-system experimental evaluation. QA = quality-assessment score (out of 8; inclusion threshold ≥6). Lifecycle codes: D = Design, Dp = Deploy, O = Operate, Op = Optimize, E = Evolve.

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S1	Database as a Service: Challenges and solutions for privacy and… [32]	2009	S	Security/Privacy	IEEE APSCC	6	O
S2	An in-depth comparison of subgraph isomorphism algorithms in gr… [111]	2012	E	Graph Databases	PVLDB	6.5	Op
S3	A Survey of Cloud Database Systems [39]	2014	S	NoSQL/NewSQL	IEEE IT Professional	6	D O Op
S4	A Systematic Review of Cloud Computing, Big Data and Databases … [40]	2014	S	DBaaS	AMCIS	8	D Dp O Op E
S5	Confidential database-as-a-service approaches: taxonomy and sur… [33]	2015	S	Security/Privacy	J. Cloud Comput.	7.5	D O Op
S6	Choosing the right NoSQL database for the job: a quality attrib… [42]	2015	S	NoSQL/NewSQL	J. Big Data	7.5	D O Op
S7	Database-as-a-Service for Big Data: An Overview [112]	2016	S	DBaaS	IJACSA	7	D Dp O Op
S8	Some problems on graph databases [113]	2016	S	Graph Databases	Proc. ISP RAS	7	Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S9	Cloud Migration Process: A Survey [82]	2016	S	Cloud DB Migration	J. Syst. Softw.	7.5	Dp E
S10	Foundations of Query Languages for Graph DBs [8]	2017	S	Graph Databases	ACM Comp. Surv.	8	D Op
S11	Open Source Time Series Databases: A Survey [52]	2017	S	Time-Series DBs	BTW	6	D O
S12	Enforcing Privacy in Cloud Databases [114]	2017	S	Security/Privacy	arXiv	6.5	O Op
S13	Big data storage technologies: a survey [115]	2017	S	NoSQL/NewSQL	Front. Inf. Technol. Electron. Eng.	7.5	D O
S14	Evolutionary Algorithms for Query Op-timization in Distributed … [116]	2018	S	AI4DB/Learned Opt.	ADCAIJ	6	Op
S15	A Survey on NoSQL Stores [11]	2018	S	NoSQL/NewSQL	ACM Comp. Surv.	8	D Dp
S16	Saving Large Semantic Data in Cloud: A Survey of the Main DBaaS… [117]	2018	S	DBaaS	Informatica Economica	6	D Dp
S17	A Survey on Graph Database Management Techniques for Huge Unstr… [118]	2018	S	Graph Databases	IJECE	7	D Op
S18	DBaaS Multitenancy, Auto-tuning and SLA Maintenance in Cloud En… [119]	2018	S	DBaaS	iSys	7	Dp O Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S19	Recent NL Interfaces for Databases: A Comparative Survey [103]	2019	S	NL/LLM-Database	VLDB J.	7.5	Op
S20	Consistency models in distributed systems: A survey on definiti… [120]	2019	S	Distributed DBs	arXiv	6	O Op
S21	Frameworks for Querying Databases Using Natural Language: A Lit… [121]	2019	S	NL/LLM-Database	arXiv	6.5	Op
S22	Architecture, Security & Performance of DBaaS [77]	2019	S	Security/Privacy	Trans. Emerg. Telecom. Tech.	6.5	D O
S23	A comparative analysis of adaptive consistency approaches in cl… [122]	2019	S	Distributed DBs	J. Parallel Distrib. Comput.	7.5	O Op
S24	Multi-Model Databases: A New Journey [53]	2019	S	Multi-Model DBs	ACM Comp. Surv.	8	D Dp
S25	LSM-based storage techniques: a survey [123]	2019	S	NoSQL/NewSQL	VLDB J.	7.5	D Op
S26	The Query Translation Landscape: a Survey [124]	2019	S	Multi-Model DBs	arXiv	6.5	Op
S27	Blockchains vs. Distributed Databases: Dichotomy and Fusion [125]	2019	S	Distributed DBs	arXiv	6	D O Op
S28	NoSQL Databases: Yearning for Disambiguation [126]	2020	S	NoSQL/NewSQL	arXiv	6	D O

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S29	Benchmarking Graph Data Management and Processing Systems: A Su… [127]	2020	S	Benchmarking	arXiv	6.5	Op
S30	SQL, NewSQL, and NoSQL Databases: A Comparative Survey [50]	2020	S	NoSQL/NewSQL	IEEE ICICS	6	D Dp
S31	Real-time analytics, hybrid transactional/analytical processing… [128]	2020	S	HTAP Systems	Proc. ISP RAS	6.5	D Op
S32	Benchmarking learned indexes [129]	2020	E	AI4DB/Learned Opt.	PVLDB	7.5	Op
S33	Querying in the Age of Graph Databases and Knowledge Graphs [130]	2021	S	Graph Databases	ACM SIGMOD Rec.	6.5	D Op
S34	Big Data Storage Tools Using NoSQL Databases and Their Applicat… [131]	2021	S	NoSQL/NewSQL	Computing and Informatics	8	D O Op E
S35	Evaluation of Distributed Databases in Hybrid Clouds and Edge C… [132]	2021	E	Edge/Fog DBs	arXiv (Cornell University)	6.5	D Dp Op
S36	Comparison and evaluation of state-of-the-art LSM merge policies [133]	2021	E	NoSQL/NewSQL	VLDB J.	7.5	Op
S37	Constructing and analyzing the LSM compaction design space [134]	2021	E	NoSQL/NewSQL	PVLDB	7.5	D Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S38	The Forgotten Document-Oriented Database Management Systems: An… [135]	2021	S	NoSQL/NewSQL	Big Data Research	7.5	D Op
S39	Integrity Authentication for SQL Query Evaluation on Outsourced… [136]	2021	S	Security/Privacy	IEEE TKDE	7.5	O
S40	Towards Polyglot Data Stores – Overview and Open Research Ques… [137]	2022	S	Multi-Model DBs	arXiv	6.5	D Op
S41	From Data Warehouse to Lakehouse: A Comparative Review [10]	2022	S	Data Lakehouse	IEEE BigData	6.5	D Dp
S42	A Survey of Comparison Different Cloud Database Performance: SQ… [138]	2022	S	NoSQL/NewSQL	Passer J.	7	D Dp O Op
S43	Database Meets AI: A Survey [9]	2022	S	AI4DB/Learned Opt.	IEEE TKDE	8	Op E
S44	Consistency issue and related trade-offs in distributed replica… [139]	2023	S	Distributed DBs	Radioelectron. Comput. Syst.	7	D O Op
S45	Demystifying Graph Databases: Analysis and Taxonomy of Data Org… [140]	2023	S	Graph Databases	ACM Comput. Surv.	8	D O Op
S46	Multidimensional Modelling in NoSQL Database : A Systematic Rev… [100]	2023	S	NoSQL/NewSQL	CISTI	7.5	D

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S47	Multi-model query languages: taming the variety of big data [141]	2023	S	Multi-Model DBs	Distrib. Parallel Databases	7.5	D Op
S48	SQL and NoSQL Database Software Architecture Performance Analys… [142]	2023	S	NoSQL/NewSQL	Big Data Cogn. Comput.	8	D Dp Op
S49	A Survey on Property-Preserving Database Encryption Techniques … [143]	2023	S	Security/Privacy	arXiv	6.5	D O Op
S50	A Comprehensive Survey on Vector Database: Storage and Retrieva… [144]	2023	S	Vector Databases	arXiv	7	D Op O
S51	The World of Graph Databases from An Industry Perspective [145]	2023	S	Graph Databases	ACM SIGMOD Rec.	7.5	D O Op
S52	An Overview on Cloud Distributed Databases for Business Environ… [146]	2023	S	DBaaS	arXiv	6	D Dp O Op
S53	Learned Index: A Comprehensive Experimental Evaluation [94]	2023	E	AI4DB/Learned Opt.	PVLDB	7	Op
S54	Understanding Graph Databases: A Comprehensive Tutorial and Sur… [147]	2024	S	Graph Databases	arXiv	6	D Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S55	Energy Consumption and Performance Evaluation of Multi-Model No… [148]	2024	E	Benchmarking	R. Inform. Teor. Apl.	7	D Op
S56	Retrieval-Augmented Generation for LLMs: A Survey [149]	2024	S	NL/LLM-Database	arXiv	7	Dp O
S57	Databases in Edge and Fog Environments: A Survey [13]	2024	S	Edge/Fog DBs	ACM Comp. Surv.	7.5	D Dp O
S58	HTAP Databases: A Survey [7]	2024	S	HTAP Systems	arXiv	6.5	D O Op
S59	The evolution of data storage architectures: examining the secu… [150]	2024	S	Data Lakehouse	J. Data Inf. Manag.	7.5	D O E
S60	Cloud-Native Databases: A Survey [6]	2024	S	Cloud-Native Arch.	IEEE TKDE	8	D Dp O Op
S61	A survey of LSM-Tree based Indexes, Data Systems and KV-stores [151]	2024	S	NoSQL/NewSQL	IEEE SCEECS	7.5	D Op
S62	A Systematic Review of Automated Classification for Simple and … [152]	2024	S	AI4DB/Learned Opt.	Comput. Syst. Sci. Eng.	7.5	Op O
S63	Survey of Vector Database Management Systems [5]	2024	S	Vector Databases	VLDB J.	7.5	D Dp Op
S64	Data Lakehouse for Time Series Data: An SLR [55]	2024	S	Data Lakehouse	IEEE BigData	6.5	D O

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S65	Data Modeling for Connected Data – A systematic literature rev… [153]	2024	S	Graph Databases	arXiv	7	D E
S66	Sharding Distributed Databases: A Critical Review [154]	2024	S	Distributed DBs	arXiv	6	D O Op
S67	A survey on hybrid transactional and analytical processing [155]	2024	S	HTAP Systems	VLDB J.	7.5	D Op
S68	DBMS Performance Comparisons: An SLR [15]	2024	S	Benchmarking	J. Syst. Softw.	7.5	Op
S69	Vector database management systems: Fundamental concepts, use-c… [156]	2024	S	Vector Databases	Cogn. Syst. Res.	7.5	D O Op
S70	Cassandra vs. MongoDB: A Systematic Review [87]	2024	S	NoSQL/NewSQL	IEEE BDAI	6	D Op
S71	Automatic Configuration Tuning on Cloud Database: A Survey [157]	2024	S	AI4DB/Learned Opt.	arXiv	6.5	Op O
S72	A Survey of Learned Indexes for the Multi-dimensional Space [158]	2025	S	AI4DB/Learned Opt.	ACM Comput. Surv.	7.5	D Op
S73	Survey: On the Landscape of Graph Databases [24]	2025	S	Graph Databases	arXiv	7	D Dp O Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S74	Evaluating Performance and User Perceptions of Multi-Cloud Data… [71]	2025	S	DBaaS	IEEE ICAC	6.5	Dp O Op
S75	When Large Language Models Meet Vector Databases: A Survey [159]	2025	S	NL/LLM-Database	IEEE AIxMM	7	D Op
S76	NoSQL Databases: A Survey [12]	2025	S	NoSQL/NewSQL	ARIMA	7.5	D Dp
S77	Graph Neural Networks for Databases: A Survey [160]	2025	S	AI4DB/Learned Opt.	arXiv	6.5	Op
S78	Evaluating Learned Indexes in LSM-tree Systems: Benchmarks,Insi… [161]	2025	E	AI4DB/Learned Opt.	arXiv	6.5	Op
S79	How good are multi-dimensional learned indexes? An experimental… [162]	2025	E	Benchmarking	VLDB J.	7.5	Op
S80	Survey on Cloud Database Security: Cryptographic Techniques, In… [163]	2025	S	Security/Privacy	EPJ Web Conf.	7.5	O Op
S81	NoSQL Data Warehouse Optimizing Models [99]	2025	S	NoSQL/NewSQL	Big Data Research	6	D Op
S82	Learning database optimization techniques: the state-of-the-art… [164]	2025	S	AI4DB/Learned Opt.	Front. Comput. Sci.	7.5	Op

Continued on next page

ID	Title (Abbreviated)	Year	Type	Primary Topic	Venue	QA	Lifecycle
S83	Evaluating the Performance of NoSQL Databases for Big Data in C… [165]	2025	E	NoSQL/NewSQL	HighTech Innov. J.	7.5	Dp Op
S84	Toward Understanding Bugs in Vector Database Management Systems [166]	2025	E	Vector Databases	arXiv	7	Dp O Op

References

[1] DB-Engines, DB-Engines ranking of database management systems, https://db-engines.com/en/ranking, accessed: 2026-07-05 (2026).

[2] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, X. Bao, Amazon Aurora: Design considerations for high throughput cloud-native relational databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2017, pp. 1041–1052. doi:10.1145/3035918.3056101.

[3] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, P. Unterbrunner, The Snowflake elastic data warehouse, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 215–226. doi:10.1145/2882903.2903741.

[4] P. Antonopoulos, A. Budovski, C. Diaconu, A. H. Saenz, J. Hu, H. Kodavalla, D. Kossmann, S. Lingam, U. F. Minhas, N. Prakash, V. Purber, H. Qu, C. S. Ravella, K. Reisteter, S. Shrotri, D. Tang, V. Wakade, Socrates: The new SQL server in the cloud, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2019, pp. 1743–1756. doi:10.1145/3299869.3314047.

[5] J. J. Pan, J. Wang, G. Li, Survey of vector database management systems, The VLDB Journal 33 (5) (2024) 1371–1400. doi:10.1007/s00778-024-00864-x.

[6] H. Dong, C. Zhang, G. Li, H. Zhang, Cloud-native databases: A survey, IEEE Transactions on Knowledge and Data Engineering 36 (12) (2024) 7772–7791. doi:10.1109/TKDE.2024.3397508.

[7] C. Zhang, G. Li, J. Zhang, X. Zhang, J. Feng, HTAP databases: A survey, arXiv preprint arXiv:2404.15670 (2024). arXiv:2404.15670.

[8] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, D. Vrgoc, Foundations of modern query languages for graph databases, ACM Computing Surveys 50 (5) (2017) 1–40. doi:10.1145/3104031.

[9] X. Zhou, C. Chai, G. Li, J. Sun, Database meets artificial intelligence: A survey, IEEE Transactions on Knowledge and Data Engineering 34 (3) (2022) 1096–1116. doi:10.1109/TKDE.2020.2994641.

[10] A. A. Harby, F. Zulkernine, From data warehouse to lakehouse: A comparative review, in: 2022 IEEE International Conference on Big Data (Big Data), IEEE, 2022, pp. 389–395. doi:10.1109/BigData55660.2022.10020719.

[11] A. Davoudian, L. Chen, M. Liu, A survey on NoSQL stores, ACM Computing Surveys 51 (2) (2018) 1–43. doi:10.1145/3158661.

[12] D. Kpekpassi, D. Faye, NoSQL databases: A survey, Revue Africaine de Recherche en Informatique et Mathématiques Appliquées (ARIMA) (2025). doi:10.46298/arima.13970.

[13] L. M. M. Ferreira, F. Coelho, J. Pereira, Databases in edge and fog environments: A survey, ACM Computing Surveys 56 (11) (2024) 1–40. doi:10.1145/3666001.

[14] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, Technical Report EBSE-2007-01, Keele University and Durham University (2007).

[15] T. Taipalus, Database management system performance comparisons: A systematic literature review, Journal of Systems and Software 208 (2024) 111872. doi:10.1016/j.jss.2023.111872.

[16] Gartner, Inc., Gartner says the future of the database market is the cloud, https://www.gartner.com/en/newsroom/press-releases/2019-07-01-gartner-says-the-future-of-the-database-market-is-the, accessed: 2026-07-05 (2019).

[17] M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia, Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, in: Proceedings of CIDR, 2021.

[18] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Biber, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, P. Mattis, CockroachDB: The resilient geo-distributed SQL database, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2020, pp. 1493–1509. doi:10.1145/3318464.3386134.

[19] W. Cao, Y. Zhang, X. Yang, F. Li, S. Wang, Q. Hu, X. Cheng, Z. Chen, Z. Liu, J. Fang, B. Wang, Y. Wang, H. Sun, Z. Yang, Z. Cheng, S. Chen, J. Wu, W. Hu, J. Zhao, Y. Gao, S. Cai, Y. Zhang, J. Tong, PolarDB serverless: A cloud native database for disaggregated data centers, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 2477–2489doi:10.1145/3448016.3457560.

[20] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, X. Tang, TiDB: A Raft-based HTAP database, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3072–3084. doi:10.14778/3415478.3415535.

[21] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Lin, H. Xu, G. Li, L. Ran, S. Wang, Y. Li, R. Pan, L. Guo, OceanBase: A 707 million tpmC distributed relational database system, in: Proceedings of the VLDB Endowment, Vol. 15, 2022, pp. 3385–3397. doi:10.14778/3554821.3554830.

[22] F. Färber, S. K. Cha, J. Primsch, C. Bornhövd, S. Siber, W. Lehner, SAP HANA database: Data management for modern business applications, in: ACM SIGMOD Record, Vol. 40, 2012, pp. 45–51. doi:10.1145/2094114.2094126.

[23] M. Elbattah, M. Roushdy, M. Aref, A.-B. M. Salem, Large-scale ontology storage and query using graph database-oriented approach: The case of Freebase, in: 2015 IEEE Seventh International Conference on Intelligent Computing and Information Systems (ICICIS), IEEE, 2015.

[24] M. E. Coimbra, L. Svitáková, D. Vrgoč, A. P. Francisco, L. Veiga, Survey: On the landscape of graph databases, arXiv:2505.24758 (2025). doi:10.48550/arxiv.2505.24758.

[25] G. Xiao, D. Calvanese, R. Kontchakov, D. Lembo, A. Poggi, R. Rosati, M. Zakharyaschev, Ontology-based data access: A survey, Proceedings of the International Joint Conference on Artificial Intelligence (2018) 5511–5519.

[26] A. Halevy, F. Korn, N. F. Noy, C. Olston, N. Polyzotis, S. Roy, S. E. Whang, Goods: Organizing Google's datasets, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 795–806. doi:10.1145/2882903.2903730.

[27] V. Leis, M. Kuschewski, Towards cost-optimal query processing in the cloud, in: Proceedings of the VLDB Endowment, Vol. 14, 2021, pp. 1606–1612. doi:10.14778/3461535.3461549.

[28] V. Mateljan, D. Čišić, D. Ogrizović, Cloud database-as-a-service (DaaS) – ROI, in: Proceedings of the 33rd International Convention MIPRO, IEEE, 2010.

[29] I. Müller, R. Marroquín, G. Alonso, Lambada: Interactive data analytics on cold data using serverless cloud infrastructure, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2020, pp. 115–130. doi:10.1145/3318464.3389758.

[30] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation), Official Journal of the European Union (2016).

[31] E. Politou, E. Alepis, C. Patsakis, Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions, Journal of Cybersecurity 4 (1) (2018) tyy001. doi:10.1093/cybsec/tyy001.

[32] E. Ferrari, Database as a service: Challenges and solutions for privacy and security, in: 2009 IEEE Asia-Pacific Services Computing Conference (APSCC), IEEE, 2009, pp. 46–51. doi:10.1109/APSCC.2009.5394141.

[33] J. Köhler, K. Jünemann, H. Hartenstein, Confidential database-as-a-service approaches: taxonomy and survey, Journal of Cloud Computing 4 (1) (2015). doi:10.1186/s13677-014-0025-1.

[34] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, R. E. Gruber, Bigtable: A distributed storage system for structured data, ACM Transactions on Computer Systems 26 (2) (2008) 1–26. doi:10.1145/1365815.1365816.

[35] S. Ramanathan, S. Goel, S. Alagumalai, Comparison of cloud database: Amazon's SimpleDB and Google's Bigtable, in: 2011 International Conference on Recent Trends in Information Systems (ReTIS), 2011. doi:10.1109/ReTIS.2011.6146861.

[36] T. Dory, B. Mejías, P. Van Roy, N.-L. Tran, Measuring elasticity for cloud databases, in: CLOUD COMPUTING 2011: The Second International Conference on Cloud Computing, GRIDs, and Virtualization, IARIA, 2011.

[37] R. Cattell, Scalable SQL and NoSQL data stores, ACM SIGMOD Record 39 (4) (2011) 12–27. doi:10.1145/1978915.1978919.

[38] J. Han, H. E, G. Le, J. Du, Survey on NoSQL database, in: 2011 6th International Conference on Pervasive Computing and Applications (ICPCA), IEEE, 2011, pp. 363–366.

[39] G. C. Deka, A survey of cloud database systems, IT Professional 16 (2) (2014) 50–57. doi:10.1109/MITP.2013.1.

[40] A. T. Litchfield, J. Althouse, A systematic review of cloud computing, big data and databases on the cloud, in: AMCIS, 2014.
URL https://aisel.aisnet.org/amcis2014/ServiceSystems/GeneralPresentations/1

[41] I. Arora, A. Gupta, Cloud databases: A paradigm shift in databases, International Journal of Computer Science Issues 9 (4) (2012).

[42] J. R. Lourenço, B. Cabral, P. Carreiro, M. Vieira, J. Bernardino, Choosing the right nosql database for the job: a quality attribute evaluation, Journal of Big Data 2 (1) (2015). doi:10.1186/s40537-015-0025-0.

[43] C. Curino, E. P. Jones, R. A. Popa, N. Malviya, E. Wu, S. Madden, H. Balakrishnan, N. Zeldovich, Relational cloud: A database-as-a-service for the cloud, in: Proceedings of CIDR, 2011.

[44] W. Lehner, K.-U. Sattler, Database as a service (DBaaS), in: 2010 IEEE 26th International Conference on Data Engineering (ICDE), IEEE, 2010, pp. 1216–1217.

[45] W. A. Shehri, Cloud database database as a service, International Journal of Database Management Systems (IJDMS) 5 (2) (2013) 1–12. doi:10.5121/ijdms.2013.5201.

[46] K. Grolinger, W. A. Higashino, A. Tiwari, M. A. M. Capretz, Data management in cloud environments: NoSQL and NewSQL data stores, Journal of Cloud Computing: Advances, Systems and Applications 2 (1) (2013) 22. doi:10.1186/2192-113X-2-22.

[47] A. Pavlo, G. Angulo, J. Arulraj, H. Lin, J. Lin, L. Ma, P. Menon, T. C. Mowry, M. Perron, I. Quah, S. Santurkar, A. Tomasic, S. Toor, D. V. Aken, Z. Wang, Y. Wu, R. Xian, T. Zhang, Self-driving database management systems, in: Proceedings of CIDR, 2017.

[48] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, D. Woodford, Spanner: Google's globally distributed database, ACM Transactions on Computer Systems 31 (3) (2013) 1–22. doi:10.1145/2491245.

[49] F. Gessert, W. Wingerath, S. Friedrich, N. Ritter, NoSQL database systems: a survey and decision guidance, Computer Science – Research and Development 32 (3–4) (2017) 353–365. doi:10.1007/s00450-016-0334-3.

[50] T. N. Khasawneh, M. H. AL-Sahlee, A. A. Safia, SQL, NewSQL, and NoSQL databases: A comparative survey, in: 2020 11th International Conference on Information and Communication Systems (ICICS), IEEE, 2020, pp. 013–021. doi:10.1109/ICICS49469.2020.239513.

[51] B. G. Tudorica, C. Bucur, A comparison between several NoSQL databases with comments and notes, in: 2011 RoEduNet International Conference 10th Edition: Networking in Education and Research, IEEE, 2011.

[52] A. Bader, O. Kopp, M. Falkenthal, Survey and comparison of open source time series databases, in: Datenbanksysteme für Business, Technologie und Web (BTW 2017) – Workshopband, Lecture Notes in Informatics (LNI), Gesellschaft für Informatik, 2017.
URL https://dl.gi.de/handle/20.500.12116/922

[53] J. Lu, I. Holubová, Multi-model databases: A new journey to handle the variety of data, ACM Computing Surveys 52 (3) (2019) 1–38. doi:10.1145/3323214.

[54] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis, H. Ahmadi, D. Delorey, S. Min, M. Pasumansky, J. Shute, Dremel: A decade of interactive SQL analysis at web scale, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3461–3472. doi:10.14778/3415478.3415568.

[55] M. Pohl, D. Staegemann, K. Turowski, Data lakehouse for time series data: A systematic literature review, in: 2024 IEEE International Conference on Big Data (BigData), IEEE, 2024. doi:10.1109/BigData62323.2024.10825961.

[56] S. Salehi, A. Selamat, H. Fujita, Systematic mapping study on granular computing, Knowledge-Based Systems 80 (2015) 78–97. doi:10.1016/j.knosys.2015.02.018.

[57] G. Murtaza, L. Shuib, A. W. A. Wahab, G. Mujtaba, H. F. Nweke, M. A. Al-garadi, F. Zulfiqar, G. Raza, N. A. Azmi, Deep learning-based breast cancer classification through medical imaging modalities: State of the art and research challenges, Artificial Intelligence Review 53 (3) (2020) 1655–1720. doi:10.1007/s10462-019-09716-5.

[58] A. Qazi, H. Fayaz, A. Wadi, R. G. Raj, N. Rahim, W. A. Khan, The artificial neural network for solar radiation prediction and designing solar systems: A systematic literature review, Journal of Cleaner Production 104 (2015) 1–12. doi:10.1016/j.jclepro.2015.04.041.

[59] W. Saeed, C. Omlin, Explainable AI (XAI): A systematic meta-survey of current challenges and future opportunities, Knowledge-Based Systems 263 (2023) 110273. doi:10.1016/j.knosys.2023.110273.

[60] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al., The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, BMJ 372 (2021) n71. doi:10.1136/bmj.n71.

[61] R. J. Thara, S. Shine, C. Sanu, Optimizing the performance of database as a service (DaaS) model – a distributed approach, in: 2013 Fourth International Conference on Computing, Communications and Networking Technologies (ICCCNT), IEEE, 2013.

[62] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, G. J. Gordon, Query-based workload forecasting for self-driving database management systems, Proceedings of the ACM SIGMOD International Conference on Management of Data (2018) 631–645doi:10.1145/3183713.3196908.

[63] E. Brewer, CAP twelve years later: How the "rules" have changed, Computer 45 (2) (2012) 23–29. doi:10.1109/MC.2012.37.

[64] D. J. Abadi, Consistency tradeoffs in modern distributed database system design: CAP is only part of the story, Computer 45 (2) (2012) 37–42. doi:10.1109/MC.2012.33.

[65] P. Viotti, M. Vukolić, Consistency in non-transactional distributed storage systems, ACM Computing Surveys 49 (1) (2016) 1–34. doi:10.1145/2926965.

[66] T. Rabl, M. Danisch, M. Frank, S. Schindler, H.-A. Jacobsen, Just can't get enough—synthesizing big data benchmarks, IEEE BigData Congress (2019).

[67] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 1275–1288doi:10.1145/3448016.3452838.

[68] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis, The case for learned index structures, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2018, pp. 489–504. doi:10.1145/3183713.3196909.

[69] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, Z. Li, An end-to-end automatic cloud database tuning system using deep reinforcement learning, Proceedings of the ACM SIGMOD International Conference on Management of Data (2019) 415–432doi:10.1145/3299869.3300085.

[70] M. T. Samarta, A. A. S. Gunawan, M. E. Syahputra, Systematic literature review and comparative performance analysis of SQL and NoSQL databases in big data applications, in: 2024 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS), IEEE, 2024. doi:10.1109/ICIMCIS63449.2024.10957463.

[71] D. M. P. D. Daundasekara, et al., Evaluating performance and user perceptions of multi-cloud database-as-a-service solutions: A mixed-methods study, in: 2025 International Conference on Computing, 2025. doi:10.1109/ICAC69156.2025.11361452.

[72] B. Furht, A. Escalante, Handbook of cloud computing, Springer (2010). doi:10.1007/978-1-4419-6524-0.

[73] Z. S. Zubi, On distributed database security aspects, in: 2009 International Conference on Multimedia Computing and Systems (ICMCS), IEEE, 2009.

[74] V. Waghmare, K. Gojre, A. Watpade, Approach to enhancing concurrent and self-reliant access to cloud database: A review, in: 2015 International Conference on Computational Intelligence and Communication Networks (CICN), IEEE, 2015. doi:10.1109/CICN.2015.158.

[75] B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, M. Ayaz, A systematic literature review on cloud computing security: Threats and mitigation strategies, IEEE Access 9 (2021) 57792–57807. doi:10.1109/ACCESS.2021.3073203.

[76] K. Munir, Security model for cloud database as a service (DBaaS), in: 2015 International Conference on Cloud Technologies and Applications (CloudTech), IEEE, 2015.

[77] F. A. Khan, M. Jamjoom, A. Ahmad, M. Asif, An analytic study of architecture, security, privacy, query processing, and performance evaluation of database-as-a-service, Transactions on Emerging Telecommunications Technologies 30 (12) (2019) e3814. doi:10.1002/ett.3814.

[78] A. Acar, H. Aksu, A. S. Uluagac, M. Conti, A survey on homomorphic encryption schemes: Theory and implementation, ACM Computing Surveys 51 (4) (2018) 1–35. doi:10.1145/3214303.

[79] R. R. Ravan, N. B. Idris, Z. Mehrabani, A survey on querying encrypted data for database as a service, in: 2013 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC), IEEE, 2013. doi:10.1109/CyberC.2013.12.

[80] L. Ferretti, M. Colajanni, M. Marchetti, Distributed, concurrent, and independent access to encrypted cloud databases, IEEE Transactions on Parallel and Distributed Systems 25 (2) (2014) 437–446. doi:10.1109/TPDS.2013.154.

[81] A. Cuzzocrea, C. Mastroianni, G. M. Grasso, Private databases on the cloud: Models, issues and research perspectives, in: 2016 IEEE International Conference on Big Data (Big Data), 2016, pp. 3656–3661. doi:10.1109/BigData.2016.7841032.

[82] M. F. Gholami, F. Daneshgar, G. Low, G. Beydoun, Cloud migration process—a survey, evaluation framework, and open challenges, Journal of Systems and Software 120 (2016) 31–69. doi:10.1016/j.jss.2016.06.068.

[83] V. Narasayya, S. Das, M. Syamala, B. Chandramouli, S. Chaudhuri, SQLVM: Performance isolation in multi-tenant relational database-as-a-service, Proceedings of CIDR (2015).

[84] S. Jain, D. Moritz, B. Howe, High variety cloud databases, in: 2016 IEEE 32nd International Conference on Data Engineering Workshops (ICDEW), 2016, pp. 12–19. doi:10.1109/ICDEW.2016.7495609.

[85] E. Baralis, A. D. Valle, P. Garza, C. Rossi, F. Scullino, SQL versus NoSQL databases for geospatial applications, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 3388–3397.

[86] K. Mahmood, K. Orsborn, T. Risch, Comparison of NoSQL datastores for large scale data stream log analytics, in: 2019 IEEE International Conference on Smart Computing (SMARTCOMP), IEEE, 2019. doi:10.1109/SMARTCOMP.2019.00093.

[87] H. Tu, Cassandra vs. MongoDB: A systematic review of two NoSQL data stores in their industry uses, in: 2024 IEEE 7th International Conference on Big Data and Artificial Intelligence (BDAI), IEEE, 2024. doi:10.1109/BDAI62182.2024.10692676.

[88] W. Hendricks, Review of NoSQL data stores: Using a reactive three-tier application for software developers to achieve a high availability application design architecture, in: 2019 International Conference (IEEE), IEEE, 2019.

[89] N. Tripathi, NoSQL database education: A review of models, tools and teaching methods, Journal of Systems and Software 226 (2025) 112391. doi:10.1016/j.jss.2025.112391.

[90] M. Sheng, S. Wang, Y. Zhang, K. Wang, J. Wang, Y. Luo, R. Hao, MQRLD: A multimodal data retrieval platform with query-aware feature representation and learned index based on data lake, Information Processing & Management 62 (5) (2025) 104101. doi:10.1016/j.ipm.2025.104101.

[91] C. Pahl, P. Jamshidi, O. Zimmermann, Architectural principles for cloud software, ACM Transactions on Internet Technology 18 (2) (2018) 1–23. doi:10.1145/3104028.

[92] T. A. M. Phan, J. K. Nurminen, M. Di Francesco, Cloud databases for internet-of-things data, in: 2014 IEEE International Conference on Internet of Things (iThings), 2014, pp. 117–124. doi:10.1109/iThings.2014.26.

[93] X. Zhao, K.-Y. Lam, Density based learned spatial index for clustered data, Information Systems 135 (2026) 102606. doi:10.1016/j.is.2025.102606.

[94] Z. Sun, X. Zhou, G. Li, Learned index: A comprehensive experimental evaluation, Proceedings of the VLDB Endowment 16 (8) (2023) 1992–2004. doi:10.14778/3594512.3594528.

[95] J. Tan, T. Zhang, F. Li, J. Chen, Q. Zheng, P. Zhang, H. Qiao, Y. Shi, W. Cao, R. Zhang, iBTune: Individualized buffer tuning for large-scale cloud databases, in: Proceedings of the VLDB Endowment, Vol. 12, 2019, pp. 1221–1234. doi:10.14778/3339490.3339503.

[96] B. Hilprecht, C. Binnig, U. Röhm, Learning a partitioning advisor for cloud databases, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 143–157. doi:10.1145/3318464.3389704.

[97] E. Dritsas, M. Trigka, Machine learning in modern database systems: Techniques, architectures, and deployment challenges, Engineering Applications of Artificial Intelligence 170 (2026) 114225. doi:10.1016/j.engappai.2026.114225.

[98] International Energy Agency, Data centres and data transmission networks, https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks, accessed: 2026-07-05 (2024).

[99] M. Mouhibha, A. Mabrouk, Nosql data warehouse optimizing models: A comparative study of column-oriented approaches, Big Data Research 40 (2025) 100523. doi:10.1016/j.bdr.2025.100523.

[100] L. M. Ferreira, et al., Multidimensional modelling in NoSQL database: A systematic review, in: 2023 18th Iberian Conference on Information Systems and Technologies (CISTI), IEEE, 2023. doi:10.23919/CISTI58278.2023.10211592.

[101] M. Mouhibha, A. Mabrouk, An empirically-driven clustering framework for NoSQL data warehouse conversion: Optimizing column family design from relational big data using HDBSCAN, Information and Software Technology 195 (2026) 108117. doi:10.1016/j.infsof.2026.108117.

[102] M. Ma, Z. Yin, S. Zhang, S. Wang, C. Zheng, X. Jiang, H. Hu, C. Luo, Y. Li, N. Qiu, F. Li, C. Chen, D. Pei, Diagnosing root causes of intermittent slow queries in cloud databases, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 1176–1189. doi:10.14778/3389133.3389136.

[103] K. Affolter, K. Stockinger, A. Bernstein, A comparative survey of recent natural language interfaces for databases, The VLDB Journal 28 (5) (2019) 793–819. doi:10.1007/s00778-019-00567-8.

[104] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, Advances in Neural Information Processing Systems 36 (2023).

[105] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, Proceedings of the VLDB Endowment 17 (5) (2024) 1132–1145. doi:10.14778/3641204.3641221.

[106] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, Z. Li, MAC-SQL: A multi-agent collaborative framework for text-to-SQL, arXiv preprint arXiv:2312.11242 (2024).

[107] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 3911–3921. doi:10.18653/v1/D18-1425.

[108] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, G. Zeng, D-bot: Database diagnosis system using large language models, in: Proceedings of the VLDB Endowment, Vol. 17, 2024, pp. 2514–2527. doi:10.14778/3675034.3675043.

[109] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang, G. Wei, W. Zhao, F. Zhou, D. Qi, H. Yi, S. Liu, F. Chen, DB-GPT: Empowering database interactions with private large language models, arXiv preprint arXiv:2312.17449 (2024).

[110] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, J. Wang, GPTuner: A manual-reading database tuning system via GPT-guided bayesian optimization, Proceedings of the VLDB Endowment 17 (8) (2024) 1939–1952. doi:10.14778/3659437.3659449.

[111] J. Lee, W.-S. Han, R. Kasperovics, J.-H. Lee, An in-depth comparison of subgraph isomorphism algorithms in graph databases, Proceedings of the VLDB Endowment 6 (2) (2012) 133–144. doi:10.14778/2535568.2448946.

[112] M. Abourezq, A. Idrissi, Database-as-a-service for big data: An overview, International Journal of Advanced Computer Science and Applications 7 (1) (2016). doi:10.14569/ijacsa.2016.070124.

[113] R. Guralnik, Some problems on graph databases, Proceedings of the Institute for System Programming of the RAS 28 (4) (2016) 193–216. doi:10.15514/ispras-2016-28(4)-12.

[114] S. S. Moghadam, J. Darmont, G. Gavin, Enforcing privacy in cloud databases, arXiv:1708.09171 (2017). doi:10.48550/arxiv.1708.09171.

[115] A. Siddiqa, A. Karim, A. Gani, Big data storage technologies: a survey, Frontiers of Information Technology & Electronic Engineering 18 (8) (2017) 1040–1070. doi:10.1631/fitee.1500441.

[116] Z. Ali, H. M. Kiran, W. Shahzad, Evolutionary algorithms for query optimization in distributed database systems: A review, ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal 7 (3) (2018) 115–128. doi:10.14201/adcaij201873115128.

[117] B. Iancu, T. M. Georgescu, Saving large semantic data in cloud: A survey of the main dbaas solutions, Informatica Economica 22 (1/2018) (2018) 5–16. doi:10.12948/issn14531305/22.1.2018.01.

[118] P. N. S., K. P, K. N. P., N. P. K. M., A survey on graph database management techniques for huge unstructured data, International Journal of Electrical and Computer Engineering (IJECE) 8 (2) (2018) 1140. doi:10.11591/ijece.v8i2.pp1140-1149.
[119] V. D. S. Segalin, C. F. Dorneles, M. A. R. Dantas, Dbaas multitenancy, auto-tuning and sla maintenance in cloud environments: a brief survey, iSys - Brazilian Journal of Information Systems 11 (2) (2018) 30–42. doi:10.5753/isys.2018.362.
[120] H. N. S. Aldin, H. Deldari, M. H. Moattar, M. R. Ghods, Consistency models in distributed systems: A survey on definitions, disciplines, challenges and applications, arXiv:1902.03305 (2019). doi:10.48550/arxiv.1902.03305.
[121] H. S. Dar, M. I. Lali, M. U. Din, K. M. Malik, S. A. C. Bukhari, Frameworks for querying databases using natural language: A literature review, arXiv:1909.01822 (2019). doi:10.48550/arxiv.1909.01822.
[122] A. Khelaifa, S. Benharzallah, L. Kahloul, R. Euler, A. Laouid, A. Bounceur, A comparative analysis of adaptive consistency approaches in cloud storage, Journal of Parallel and Distributed Computing 129 (2019) 36–49. doi:10.1016/j.jpdc.2019.03.006.
[123] C. Luo, M. J. Carey, Lsm-based storage techniques: a survey, The VLDB Journal 29 (1) (2019) 393–418. doi:10.1007/s00778-019-00555-y.
[124] M. N. Mami, D. Graux, H. Thakkar, S. Scerri, S. Auer, J. Lehmann, The query translation landscape: a survey, arXiv:1910.03118 (2019). doi:10.48550/arxiv.1910.03118.
[125] P. Ruan, T. T. A. Dinh, D. Loghin, M. Zhang, G. Chen, Q. Lin, B. C. Ooi, Blockchains vs. distributed databases: Dichotomy and fusion, arXiv:1910.01310 (2019). doi:10.48550/arxiv.1910.01310.
[126] C. Asaad, K. Baïna, M. Ghogho, Nosql databases: Yearning for disambiguation, arXiv:2003.04074 (2020). doi:10.48550/arxiv.2003.04074.
[127] M. Dayarathna, T. Suzumura, Benchmarking graph data management and processing systems: A survey, arXiv:2005.12873 (2020). doi:10.48550/arxiv.2005.12873.
[128] S. D. Kuznetsov, P. E. Velikhov, Q. Fu, Real-time analytics, hybrid transactional/analytical processing, in-memory data management, and non-volatile memory, 2020 Ivannikov Ispras Open Conference (ISPRAS) (2020) 78–90doi:10.1109/ispras51486.2020.00019.
[129] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, T. Kraska, Benchmarking learned indexes, Proceedings of the VLDB Endowment 14 (1) (2020) 1–13. doi:10.14778/3421424.3421425.
[130] M. Arenas, C. Gutierrez, J. F. Sequeda, Querying in the age of graph databases and knowledge graphs, Proceedings of the 2021 International Conference on Management of Data (2021) 2821–2828doi:10.1145/3448016.3457545.
[131] A. Faridoon, M. Imran, Big data storage tools using nosql databases and their applications in various domains: A systematic review, Computing and Informatics 40 (3) (2021) 489–521. doi:10.31577/cai_2021_3_489.
[132] Y. Mansouri, V. Prokhorenko, F. Ullah, M. A. Babar, Evaluation of distributed databases in hybrid clouds and edge computing: Energy, bandwidth, and storage consumption, arXiv:2109.07260 (2021). doi:10.48550/arxiv.2109.07260.
[133] Q. Mao, S. Jacobs, W. Amjad, V. Hristidis, V. J. Tsotras, N. E. Young, Comparison and evaluation of state-of-the-art lsm merge policies, The VLDB Journal 30 (3) (2021) 361–378. doi:10.1007/s00778-020-00638-1.
[134] S. Sarkar, D. Staratzis, Z. Zhu, M. Athanassoulis, Constructing and analyzing the lsm compaction design space, in: Proceedings of the VLDB Endowment, Vol. 14, 2021, pp. 2216–2229. doi:10.14778/3476249.3476274.
[135] C.-O. Truică, E.-S. Apostol, J. Darmont, T. B. Pedersen, The forgotten document-oriented database management systems: An overview and benchmark of native xml dodbmses in comparison with json dodbmses, Big Data Research 25 (2021) 100205. doi:10.1016/j.bdr.2021.100205.
[136] B. Zhang, B. Dong, W. H. Wang, Integrity authentication for sql query evaluation on outsourced databases: A survey, IEEE Transactions on Knowledge and Data Engineering 33 (4) (2021) 1601–1618. doi:10.1109/tkde.2019.2947061.
[137] D. Glake, F. Kiehn, M. Schmidt, F. Panse, N. Ritter, Towards polyglot data stores – overview and open research questions, arXiv:2204.05779 (2022). doi:10.48550/arxiv.2204.05779.
[138] T. Shareef, K. Sharif, B. Rashid, A survey of comparison different cloud database performance: Sql and nosql, Passer Journal of Basic and Applied Sciences 4 (1) (2022) 45–57. doi:10.24271/psr.2022.301247.1104.
[139] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko, A. Sheikh-Akbari, Consistency issue and related trade-offs in distributed replicated systems and databases: a review, Radioelectronic and Computer Systems (2) (2023) 171–179. doi:10.32620/reks.2023.2.14.
[140] M. Besta, R. Gerstenberger, E. Peter, M. Fischer, M. Podstawski, C. Barthels, G. Alonso, T. Hoefler, Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries, ACM Computing Surveys 56 (2) (2023) 1–40. doi:10.1145/3604932.
[141] Q. Guo, C. Zhang, S. Zhang, J. Lu, Multi-model query languages: taming the variety of big data, Distributed and Parallel Databases 42 (1) (2023) 31–71. doi:10.1007/s10619-023-07433-1.
[142] W. Khan, T. Kumar, C. Zhang, K. Raj, A. M. Roy, B. Luo, Sql and nosql database software architecture performance analysis and assessments—a systematic literature review, Big Data and Cognitive Computing 7 (2) (2023) 97. doi:10.3390/bdcc7020097.
[143] J. Koppenwallner, E. Schikuta, A survey on property-preserving database encryption techniques in the cloud, arXiv:2312.12075 (2023). doi:10.48550/arxiv.2312.12075.
[144] L. Ma, R. Zhang, Y. Han, S. Yu, Z. Wang, Z. Ning, J. Zhang, P. Xu, P. Li, Z. Qiao, W. Ju, C. Chen, D. Wang, K. Liu, P. Wang, P. Wang, Y. Fu, C. Liu, Y. Zhou, C.-T. Lu, A comprehensive survey on vector database: Storage and retrieval technique, challenge, arXiv:2310.11703 (2023). doi:10.48550/arxiv.2310.11703.
[145] Y. Tian, The world of graph databases from an industry perspective, ACM SIGMOD Record 51 (4) (2023) 60–67. doi:10.1145/3582302.3582320.
[146] A. Vikiru, M. Muiruri, I. Ateya, An overview on cloud distributed databases for business environments, arXiv:2301.10673 (2023).
[147] S. Anuyah, V. Bolade, O. Agbaakin, Understanding graph databases: A comprehensive tutorial and survey, arXiv:2411.09999 (2024). doi:10.48550/arxiv.2411.09999.

[148] F. Falcão, J. Moura, G. Silva, C. Araujo, E. Sousa, E. Tavares, Energy consumption and performance evaluation of multi-model nosql dbmss, Revista de Informática Teórica e Aplicada 30 (2) (2024) 132–140. doi:10.22456/2175-2745.136568.

[149] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv:2312.10997 (2024).

[150] N. Janssen, T. Ilayperuma, J. Jayasinghe, F. Bukhsh, M. Daneva, The evolution of data storage architectures: examining the secure value of the data lakehouse, Journal of Data, Information and Management 6 (4) (2024) 309–334. doi:10.1007/s42488-024-00132-1.

[151] S. Mishra, A survey of lsm-tree based indexes, data systems and kv-stores, in: 2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS), 2024, pp. 1–6. doi:10.1109/sceecs61402.2024.10482249.

[152] R. A. Kadir, E. S. M. Surin, M. R. Sarker, A systematic review of automated classification for simple and complex query sql on nosql database, Computer Systems Science and Engineering 48 (6) (2024) 1405–1435. doi:10.32604/csse.2024.051851.

[153] V. Santos, Data modeling for connected data – a systematic literature review, arXiv:2410.10081 (2024). doi:10.48550/arxiv.2410.10081.

[154] S. Solat, Sharding distributed databases: A critical review, arXiv:2404.04384 (2024). doi:10.48550/arxiv.2404.04384.

[155] H. Song, W. Zhou, H. Cui, X. Peng, F. Li, A survey on hybrid transactional and analytical processing, The VLDB Journal 33 (5) (2024) 1485–1515. doi:10.1007/s00778-024-00858-9.

[156] T. Taipalus, Vector database management systems: Fundamental concepts, use-cases, and current challenges, Cognitive Systems Research 85 (2024) 101216. doi:10.1016/j.cogsys.2024.101216.

[157] L. Zhang, M. A. Babar, Automatic configuration tuning on cloud database: A survey, arXiv:2404.06043 (2024). doi:10.48550/arxiv.2404.06043.

[158] A. Al-Mamun, H. Wu, Q. He, J. Wang, W. G. Aref, A survey of learned indexes for the multi-dimensional space, ACM Computing Surveys 58 (4) (2025) 1–37. doi:10.1145/3768575.

[159] Z. Jing, Y. Su, Y. Han, When large language models meet vector databases: A survey, in: 2025 Conference on Artificial Intelligence x Multimedia (AIxMM), 2025, pp. 7–13, arXiv:2402.01763. doi:10.1109/AIxMM62960.2025.00008.

[160] Z. Li, Y. Li, Y. Luo, G. Li, C. Zhang, Graph neural networks for databases: A survey, arXiv:2502.12908 (2025). doi:10.48550/arxiv.2502.12908.

[161] J. Liu, J. Ye, M. Chen, M. Li, S. Luo, Evaluating learned indexes in lsm-tree systems: Benchmarks, insights and design choices, arXiv:2506.08671 (2025). doi:10.48550/arxiv.2506.08671.

[162] Q. Liu, M. Li, Y. Zeng, Y. Shen, L. Chen, How good are multi-dimensional learned indexes? an experimental survey, The VLDB Journal 34 (2) (2025). doi:10.1007/s00778-024-00893-6.

[163] R. V. Mahule, S. M. Jadhav, Survey on cloud database security: Cryptographic techniques, intrusion detection and intelligent defense mechanisms, in: EPJ Web Conf., 2025. doi:10.1051/epjconf/202534101015.

[164] S.-J. Qiao, H.-L. Fan, N. Han, L. Du, Y.-H. Peng, R.-M. Tang, X. Qin, Learning database optimization techniques: the state-of-the-art and prospects, Frontiers of Computer Science 19 (12) (2025). doi:10.1007/s11704-025-41116-7.

[165] A. E. Topcu, A. M. Rmis, Y. I. Alzoubi, A. O. Çibikdiken, D. Chowdhury, E. Elbasi, M. Abdel-Maguid, S. Almadhun, Evaluating the performance of nosql databases for big data in cloud computing environments, HighTech and Innovation Journal 6 (3) (2025) 808–830. doi:10.28991/hij-2025-06-03-05.

[166] Y. Xie, X. Hou, Y. Zhao, S. Wang, K. Chen, H. Wang, Toward understanding bugs in vector database management systems, arXiv:2506.02617 (2025). doi:10.48550/arxiv.2506.02617.